

RTK CRM — Сопроводительная документация

Кейс №6, Лидеры цифровой трансформации 2026

Команда RTK CRM

Содержание

1 Аннотация	2
1.1 Состав репозитория	2
2 Функционал по техническому заданию	2
2.1 Функциональные требования	2
2.2 Нефункциональные требования	3
3 Методы обработки данных	3
4 Ограничения прототипа	4
5 Сборка и запуск	4
6 Архитектура системы	4
6.1 □ Архитектура RTK CRM	4
7 Технологический стек	13
7.1 Технологический стек	13
8 Аутентификация и авторизация	14
8.1 Аутентификация через Keycloak	14
9 Сводка по взаимодействию (GigaChat)	17
9.1 Сводка по взаимодействию через GigaChat (ФТ-6)	17
10 Безопасность и защита данных	19
10.1 Безопасность RTK CRM	19
11 Развёртывание	23
11.1 Деплой RTK CRM	23
12 Отчёт о верификации	29
12.1 Статическая верификация RTK CRM	29
13 Коды ошибок	46
13.1 Каталог кодов ошибок API	46
14 Руководство пользователя	47
14.1 Руководство пользователя RTK CRM	47
15 Руководство администратора	52
15.1 Руководство администратора RTK CRM	52
16 Вопросы жюри и ответы	57
16.1 Вопросы жюри и ответы	57

17 Сценарии работы	63
17.1 КАМ	63
17.2 Администратор	63
18 Глоссарий	63
19 Приложения	64
19.1 Скриншоты интерфейса	64
19.2 Примеры отчётов	64
19.3 Диаграммы	64

1 Аннотация

RTK CRM — система контроля взаимодействия ИТ Школы Ростелекома с вузами-партнёрами. Система переводит разрозненные коммуникации в единый процесс с фиксированными этапами, ответственными и историей изменений.

Документ описывает архитектуру, функциональность, модель данных, методы обработки данных, ограничения прототипа и порядок развёртывания. Все требования технического задания сведены в таблицу ниже с отметкой о реализации.

1.1 Состав репозитория

Артефакт	Назначение
backend/	FastAPI-сервис: API, бизнес-логика, экспорты
frontend/	SPA на React + TypeScript
infra/	Контур развёртывания Yandex Cloud
docs/	Документация, диаграммы, презентация
scripts/	Утилиты: seed, сиды, скриншоты, сборка PDF
tests/	Набор автотестов backend
render.yaml	Blueprint развёртывания Render + Vercel + Supabase

2 Функционал по техническому заданию

Таблица отмечает реализацию каждого пункта ТЗ. Отметка «Реализовано» означает, что функциональность доступна в текущей сборке и покрыта тестами (pytest, 200 тестов) либо подтверждена сквозной проверкой.

2.1 Функциональные требования

Требование	Реализация	Где в системе
ФТ-1. Kanban-доска взаимодействий	Реализовано	Раздел «Доска», перемещение карточек @dnd-kit
ФТ-2. Карточка взаимодействия	Реализовано	Drawer с задачами, комментариями, договором
ФТ-3. Настройка этапов воркфлоу	Реализовано	«Настройки» → «Этапы воркфлоу»
ФТ-4. Справочники вузов, продуктов, направлений	Реализовано	Раздел «Справочники»

Требование	Реализация	Где в системе
ФТ-5. Интеграция с LMS и CMS заказчика	Реализовано (mock-адаптер)	Вкладка «Интеграция»
ФТ-6. ИИ-сводка по взаимодействию	Реализовано	GigaChat, кнопка в карточке
ФТ-7. Отчёты и показатели	Реализовано	Раздел «Отчёты»
ФТ-8. Экспорт отчётов (XLSX, PDF, PNG, JSON)	Реализовано	Кнопки экспорта на «Отчётах»
ФТ-9. Импорт справочников из XLSX	Реализовано	«Справочники» → «Импорт»
ФТ-10. Авторизация через Keycloak	Реализовано	Кнопка «Войти через Keycloak» + PKCE
ФТ-11. Ролевая модель (RBAC)	Реализовано	Роли admin / manager / kam
ФТ-12. Задачи со сроками	Реализовано	Вкладка задач в карточке
ФТ-13. Файлы и вложения	Реализовано	S3-совместимое хранилище (MinIO / Object Storage)

2.2 Нефункциональные требования

Требование	Реализация	Подтверждение
НФТ-1. Производительность	Реализовано	Нагрузочный прогон Locust: 50 пользователей / 60 с, 1613 запросов, 0 ошибок, p95 93 мс (docs/VERIFICATION_REPORT.md)
НФТ-2. Производительность отчётов	Реализовано	Кэш агрегатов в Redis/KeyDB (backend/services/cache.py)
НФТ-3. Стабильные коды ошибок	Реализовано	docs/ERROR_CODES.md
НФТ-4. Адаптивность интерфейса	Реализовано	Мобильный вид Kanban, одна колонка
НФТ-5. Выгрузка документации в PDF	Реализовано	scripts/build_docs_pdf.py, кнопки в /help
НФТ-6. Безопасность и 152-ФЗ	Реализовано	docs/SECURITY.md, аудит, RBAC
НФТ-7. Наблюдаемость	Реализовано	/healthz, /readyz, Prometheus-метрики

3 Методы обработки данных

- **Хранение.** PostgreSQL 16 (или SQLite для локального запуска). Доступ через SQLAlchemy 2.0 в асинхронном режиме.
- **Импорт XLSX.** Разбор openpplx с диалоговым сопоставлением колонок: пользователь видит предпросмотр и подтверждает маппинг до записи. Импорт справочников идемпотентен по натуральному ключу.
- **Обмен с LMS/CMS.** Пакеты JSON проходят валидацию схемой (jsonschema) в режиме dry-run. Импорт идемпотентен по contract_number: повтор пакета обновляет запись, а не создаёт дубль.
- **Экспорт.** XLSX (openpplx), PDF и PNG (reportlab), JSON. Экспорты выгружаются с учётом прав: KAM видит только свои вузы.
- **Аудит.** Каждое изменяющее действие пишется в журнал (ActionLog) с автором, сущностью и временем; журнал доступен администратору.

- **Кэширование.** Агрегаты отчётов кэшируются в Redis/KeyDB с инвалидацией при изменении взаимодействий.

4 Ограничения прототипа

- **LMS и CMS — mock-адаптеры.** Внешние системы не подключены: адаптеры возвращают детерминированные данные по контракту обмена. Замена на реальные API выполняется реализацией того же интерфейса.
- **GigaChat.** Без ключа функция выключена: API отвечает 503, а при включённом демо-режиме (GIGACHAT_FALLBACK_ENABLED=true) возвращается локальная сводка с явной пометкой, что модель не вызывалась.
- **Ограничение бизнес-модели.** У одного вуза не более двух активных взаимодействий; проверка выполняется на backend.
- **Файлы.** В прототипе хранилище — MinIO или Yandex Object Storage; virus-скан не подключён.

5 Сборка и запуск

```
# Backend
pip install -r backend/requirements.txt
python scripts/run_dev.py --port 8000
```

```
# Frontend
cd frontend && npm ci && npm run dev
```

```
# Тесты
python -m pytest -q
ruff check backend tests
```

Полный контур через Docker Compose описан в README.md. Развёртывание на Yandex Cloud и на Render + Vercel + Supabase — в разделе «Развёртывание».

6 Архитектура системы

6.1 □ Архитектура RTK CRM

6.1.1 Обзор системы

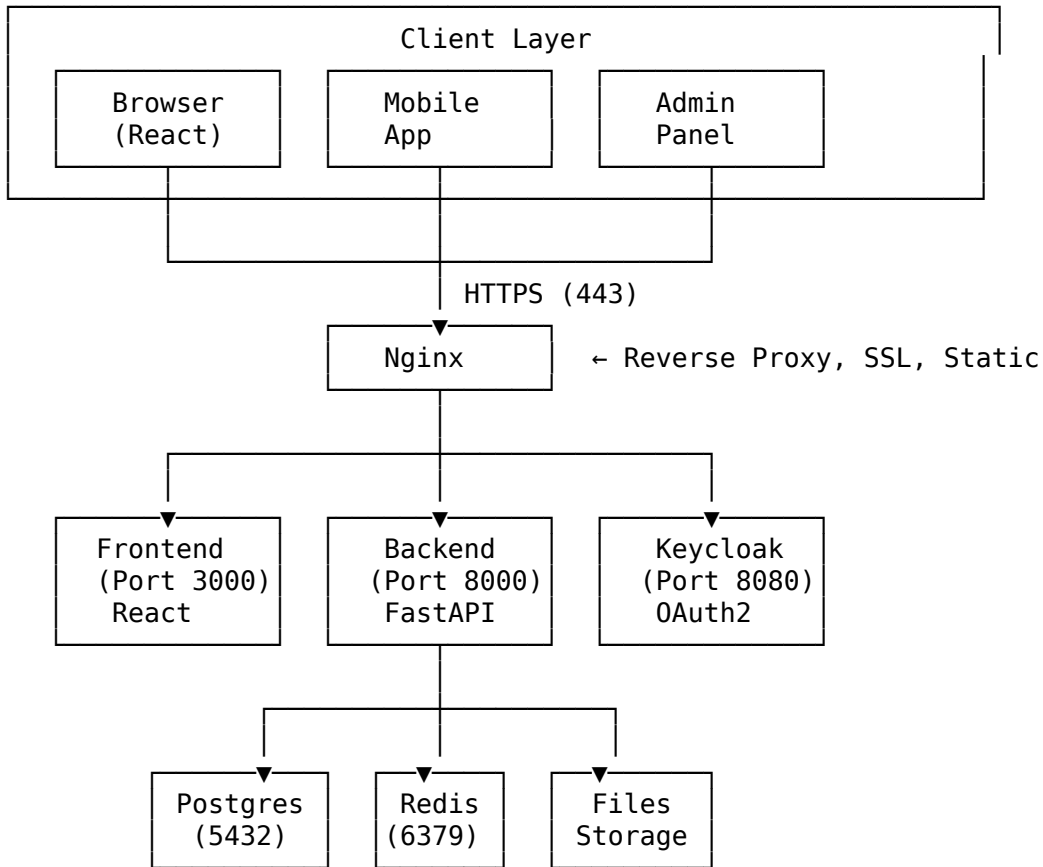
RTK CRM — это B2B CRM-система для менеджеров Ростелекома, управляющих взаимодействием с вузами. Система реализует полный цикл из 14 этапов воркфлоу.

6.1.2 Диаграммы

- **C4 Level 1** — System Context
- **C4 Level 2** — Containers / Components
- **Функциональная архитектура** — пользовательский путь и сервисы
- **Модель данных (ER)** — сущности и связи
- **ER в Archi** — er.archimate (ArchiMate 3, открывается в Archi) и PDF-версия диаграммы
- **Модель Archi** — функциональная и компонентная архитектура (ArchiMate 3, открывается в Archi)
- **Развёртывание** — текущее и в Yandex Cloud

Диаграммы написаны в Mermaid и рендерятся прямо в GitHub/GitLab, поэтому не требуют отдельного экспорта в PDF для ревью. Версия для презентации собирается из этих же исходников.

6.1.3 Диаграмма компонентов



6.1.4 Технологический стек и обоснование выбора

Полные версии зависимостей — в `../STACK.md`.

6.1.4.1 Backend

- **Язык:** Python 3.11
- **Фреймворк:** FastAPI 0.141
- **ORM:** SQLAlchemy 2.0 (AsyncIO)
- **Валидация:** Pydantic v2
- **Миграции:** Alembic
- **Аутентификация:** python-jose, passlib

6.1.4.2 Frontend

- **Фреймворк:** React 18
- **Язык:** TypeScript 5.x
- **Сборка:** Vite 7
- **UI Kit:** Ant Design
- **HTTP клиент:** Axios

6.1.4.3 Базы данных

- **Основная:** PostgreSQL 16 (ACID, JSONB, Full-text search)
- **Кэш:** Redis 7 → KeyDB при росте нагрузки (см. раздел «Масштабируемость»)

6.1.4.4 Инфраструктура

- **Контейнеризация:** Docker, Docker Compose
- **Web Server:** Nginx (Alpine) — целевой контур Yandex Cloud
- **Auth:** JWT (HS256) + bcrypt; Keycloak — вариант целевого контура

6.1.4.5 Почему именно этот стек

Компонент	Выбор	Почему не альтернатива
Backend	Python + FastAPI	T3 само предлагает Python/Postgres; FastAPI даёт асинхронный I/O под нагрузку 300+, автогенерацию OpenAPI и общий язык типов с Pydantic. Kotlin/Node потребовали бы отдельной команды и переписывания без выигрыша по задачам CRM
Frontend	React + Ant Design	Ant Design закрывает Kanban, таблицы, формы и фильтры «из коробки» — на прототипе это экономит недели вёрстки; React выбран из-за экосистемы dnd-kit (drag-and-drop этапов) и Recharts (графики отчётов)
БД	PostgreSQL	Реляционная модель с транзакциями подходит для workflow и аудита (152-ФЗ); SQLite используется только в тестах и локальном запуске
Кэш	Redis/KeyDB	Отчёты агрегируются по многим строкам; кэш на 30 с снимает нагрузку. KeyDB — многопоточный форк Redis с тем же протоколом, поэтому переход без изменений кода
Аутентификация	JWT сейчас, Keycloak как задел	Для прототипа JWT не требует отдельного сервиса; в коде оставлен путь к корпоративному Keycloak, если заказчик потребует SSO

Единственное нестандартное решение — отказ от версионирования workflow (прямое требование Q&A: «изменения применяются едино для всех»), поэтому в модели нет таблицы версий, а перенос задач при удалении этапа встроен в сервис.

6.1.5 Модульная архитектура Backend

```
backend/  
├── main.py           # Точка входа, middleware, health-пробы  
├── api/  
│   └── auth.py       # REST-роутеры (префикс /api)  
│                       # Вход, пользователи
```

— interactions.py	# Карточки, доска, задачи, комментарии, сводка
— universities.py	# Вузы
— directories.py	# Направления и продукты
— catalogs.py	# Импорт XLS/XLSX/JSON
— stages.py	# Этапы воркфлоу, последствия удаления
— files.py	# Загрузка и выдача файлов
— reports.py	# Отчёты и экспорт
— audit.py	# Журнал действий (152-ФЗ)
— notifications.py	# Зависшие заявки
— integration.py	# Обмен с внешними системами
— access.py	# Проверка доступа к взаимодействиям
— auth/	
— security.py	# JWT, пароли, get_current_user, RBAC
— keycloak.py	# JWKS-валидация RS256 (AUTH_MODE=keycloak)
— core/config.py	# Настройки (pydantic-settings)
— db/	# Подключение к БД, базовый класс, сессии
— middleware/audit.py	# Автоматическая запись изменений в ActionLog
— models/	# SQLAlchemy-модели и перечисления
— entities.py	# University, Interaction, Action, Comment, User, ActionLog
— enums.py	# Роли, scope B2B/B2C
— schemas/entities.py	# Pydantic-схемы (DTO)
— services/	# Бизнес-логика
— excel_import.py	# Разбор XLS/XLSX/JSON
— import_report.py	# Отчёт об импорте
— excel_export.py	# Выгрузка отчётов
— report_columns.py	# Набор колонок отчёта
— report_cache.py	# Кэш отчётов (Redis/KeyDB)
— file_storage.py	# S3/MinIO с локальным fallback
— notifications.py	# Поиск зависших заявок
— scheduler.py	# Периодические задачи
— integration.py	# Внешние системы (LMS)
— guide_pdf.py	# PDF-руководства
— gigachat.py	# Сводка по карточке (ФТ-6)

6.1.6 Схема базы данных

Полная ER-диаграмма с полями, связями и правилами целостности — в `er-model.md`. Кратко:

Таблица	Назначение
users	Пользователи и роли (user / manager / admin)
universities	Вузы и контактные лица
it_directions, it_products	Справочники направлений и продуктов
workflow_stages	Этапы воронки, разделены полем scope (b2b / b2c)
interactions	Карточки взаимодействий: вуз + продукт + этап + договор
actions, comments, attached_files	Задачи, комментарии и файлы карточки
action_logs	Журнал действий для аудита (152-ФЗ)

Ключевое отличие от ранних версий: этап и договор относятся к **взаимодействию**

(interactions), а не к вузу напрямую — один вуз может вести несколько параллельных взаимодействий (не более двух активных, как требует Q&A).

6.1.7 Безопасность и 152-ФЗ

6.1.7.1 RBAC (Role-Based Access Control)

Роль	Права
User	Чтение вузов, комментарии
Manager	CRUD вузов, загрузка файлов, перемещение по воркфлоу
Admin	Полный доступ, управление пользователями, логи

6.1.7.2 Логирование (ФСТЭК) Все действия записываются в `action_logs`: - Кто (`user`, `ip_address`) - Что сделал (`action`, `old_value`, `new_value`) - Когда (`created_at`) - С чем (`university_id`)

6.1.7.3 Аутентификация Два режима, переключаются `AUTH_MODE`:

- **jwt** (по умолчанию) — JWT Bearer-токены (HS256), пароли — bcrypt. Роли `user` / `manager` / `admin`, проверяются локально.
- **keycloak** — токены подписывает Keycloak (RS256), CRM проверяет подпись по публичным ключам `realm` (JWKS), затем `iss`, `aud` и срок действия. Роли берутся из `realm_access.roles` и синхронизируются с локальной учётной записью при входе. Устройство и порядок настройки — `docs/KEYCLOAK.md`.

HTTPS обязателен в обоих режимах. Сводка по карточке передаёт данные во внешний сервис GigaChat только после проверки доступа (см. `docs/AI.md`).

6.1.7.4 ИИ в workflow ТЗ допускает применение ИИ внутри workflow (ФТ-6). В системе это одна функция — сводка по карточке взаимодействия: КАМ открывает карточку и получает короткий текст о текущем этапе, сделанном и следующем шаге.

Аспект	Решение
Модель	GigaChat, закрытый контур Сбера
Данные в промпте	Только названия (вуз, продукт, этап), тексты комментариев и служебные поля
Персональные данные	Не передаются: в карточке не хранятся, отбрасываются на импорте
Соответствие	152-ФЗ: обработка идёт в РФ, данные не покидают контур
Транспорт	HTTPS, проверка по корневому сертификату НУЦ Минцифры
Сбой сервиса	Не ломает карточку: 502 при выключенном демо-режиме, детерминированная сводка из данных карточки при включённом

Демо-сводка помечается отдельной моделью демо-сводка (GigaChat не вызывался), поэтому её нельзя принять за ответ модели. Устройство и настройка — `docs/AI.md`, требования безопасности — `docs/SECURITY.md`.

6.1.8 API Endpoints

Полная схема — в Swagger UI (`/docs`). Ниже — маршруты, которые приложение регистрирует фактически; префикс `/api` у всех, кроме служебных проб.

6.1.8.1 Аутентификация и пользователи

- POST /api/auth/login — вход, выдача JWT
- GET /api/auth/me — текущий пользователь
- GET|POST /api/auth/users — список и создание пользователей
- PATCH|DELETE /api/auth/users/{user_id} — изменение и удаление

6.1.8.2 Взаимодействия

- GET|POST /api/interactions — список с фильтрами и создание
- GET /api/interactions/board — данные Kanban-доски
- GET|PATCH|DELETE /api/interactions/{interaction_id} — карточка
- POST /api/interactions/{interaction_id}/move — смена этапа
- GET|POST /api/interactions/{interaction_id}/actions — задачи
- PATCH|DELETE /api/interactions/actions/{action_id}
- GET|POST /api/interactions/{interaction_id}/comments — комментарии
- DELETE /api/interactions/comments/{comment_id}
- POST /api/interactions/{interaction_id}/summary — сводка GigaChat (ФТ-6)

6.1.8.3 Справочники и импорт

- GET|POST /api/universities, GET|PATCH|DELETE /api/universities/{university_id}
- GET|POST /api/directions, DELETE /api/directions/{direction_id}
- GET|POST /api/products, DELETE /api/products/{product_id}
- POST /api/catalogs/import/preview и /execute — импорт XLS/XLSX
- POST /api/catalogs/import/json/preview и /execute — импорт JSON
- POST /api/catalogs/import/report — отчёт об импорте

6.1.8.4 Этапы воркфлоу

- GET|POST /api/stages, POST /api/stages/reorder
- PATCH|DELETE /api/stages/{stage_id}
- GET /api/stages/{stage_id}/impact — последствия удаления этапа

6.1.8.5 Файлы

- GET /api/files/interactions/{interaction_id} — файлы карточки
- POST /api/files/interactions/{interaction_id}/upload
- GET /api/files/{file_id}/download, DELETE /api/files/{file_id}

6.1.8.6 Отчёты и аудит

- GET /api/reports, /columns, /preview, /universities
- GET /api/reports/xlsx, /xls, /pdf, /json
- GET /api/audit, /entity-types, /export

6.1.8.7 Уведомления и интеграция

- GET /api/notifications/stuck, POST /api/notifications/stuck/dispatch
- GET /api/integration/schema, POST /api/integration/preview и /import
- GET /api/integration/outbound, GET /api/integration/pull/{source}

6.1.8.8 Служебные

- GET /healthz — liveness, GET /readyz — readiness (БД, кэш, S3)
- GET /metrics — метрики Prometheus

GigaChat в /readyz намеренно не проверяется: функция необязательная, и её недоступность не должна переводить сервис в состояние degraded.

6.1.9 Масштабируемость

Целевая нагрузка — **300+ одновременных пользователей** ИТ Школы: менеджеры работают с доской и карточками весь день, администраторы параллельно грузят справочники и смотрят отчёты. Ниже — за счёт чего система держит этот профиль и что меняется при росте дальше.

6.1.9.1 Что уже заложено в код

Механизм	Где	Зачем при 300+
Полностью асинхронный стек (FastAPI + SQLAlchemy AsyncIO + asyncpg)	backend/main.py, backend/db/session.py	I/O-ожидание БД не блокирует воркер: один процесс обслуживает сотни соединений
NullPool для подключений	backend/db/session.py	Совместимо с внешним пулером (pgbouncer Yandex Managed PostgreSQL, Supabase pooler): соединения не «залипают» в процессе
Кеш отчёта в Redis/KeyDB, TTL 30 с	backend/services/report_cache.py	Тяжёлые агрегаты не считаются на каждый запрос — снимает нагрузку с PostgreSQL
Rate limiting логина (5 попыток / 60 с на IP)	backend/middleware/rate_limit.py	Защита от перебора пароля и от всплеска 401 при сканировании
Индексы на внешних ключах и датах	backend/alembic/versions/	Фильтры доски и отчётов по этапу/дате не деградируют на росте данных
Фоновые задачи в одном инстансе (max_instances=1)	backend/services/scheduler.py	Проверка «зависших» заявок не размножается при нескольких репликах

/healthz (liveness) и /readyz (readiness, проверяет БД) разделены (backend/main.py): балансировщик уводит трафик с инстанса, потерявшего БД, но не перезапускает его из-за кратковременного сбоя зависимости.

6.1.9.2 Горизонтальное масштабирование

- **Backend:** несколько реплик за балансировщиком (Docker Swarm / Kubernetes). Приложение stateless: сессии — в JWT, кеш — в Redis/KeyDB, файлы — в S3/MinIO.
- **Кеш:** Redis заменяется на **KeyDB** — многопоточный форк с тем же протоколом и клиентом redis-py. Причина: Redis однопоточный, и на 300+ пользователях один поток упирается в CPU раньше, чем в память. KeyDB использует все ядра без изменения кода — достаточно поменять образ в docker-compose.yml.
- **База:** PostgreSQL с репликой чтения для отчётов (master — запись, реплика — тяжёлые SELECT дашборда).
- **Файлы:** MinIO/S3 выносятся за пределы пода backend, локальный диск используется только как fallback при разработке (backend/services/file_storage.py).

6.1.9.3 Когда пора менять конфигурацию

Симптом	Действие
CPU backend > 70 %	Добавить реплики бэкенда (горизонтально)
Кеш-хиты высокие, но latency растёт	Перейти на KeyDB, увеличить память кеша
Отчёты конкурируют с записью	Вынести чтение на реплику PostgreSQL
Рост числа фоновых уведомлений	Вынести scheduler в отдельный воркер

6.1.9.4 Оптимизация запросов

- Индексы на часто используемых полях (этап, продукт, дата создания, внешние ключи).
- Кеширование агрегатов отчёта (Redis/KeyDB, TTL 30 с), инвалидация при мутации.
- Асинхронные операции ввода-вывода (asyncpg, aiofiles).

6.1.10 Мониторинг и логирование

- Health checks: /healthz (liveness), /readyz (readiness + БД), /api/health (расширенный статус), /health (короткий алиас для внешних платформ и health-check).
- Логи: stdout/stderr (Docker logs), структурированные через loguru.
- Метрики: prometheus-fastapi-instrumentator на /metrics.

6.1.11 Развертывание

6.1.11.1 Production чеклист

- ☐ Заменить SECRET_KEY на случайную строку
- ☐ Настроить HTTPS сертификаты
- ☐ Включить backup PostgreSQL
- ☐ Настроить мониторинг
- ☐ Ограничить доступ к админке
- ☐ Включить rate limiting

6.1.12 Контакты

Команда разработки: team@rtk-crm.ru

Хакатон: “Лидеры цифровой трансформации 2026”

Кейс №6: ИТ Школа Ростелекома

6.1.13 План миграции в Yandex Cloud

6.1.13.1 Обоснование Сейчас контур размещён за пределами РФ: Render (backend), Vercel (frontend), Supabase (PostgreSQL, регион us-west-2). Требования 152-ФЗ «О персональных данных» к локализации баз данных граждан РФ и приказ ФСТЭК России № 117 предполагают размещение в российском контуре. Yandex Cloud даёт аттестованную инфраструктуру и переносится без изменения кода — весь стек уже контейнеризован. Внешний контур при этом остаётся рабочим и используется как публичный демонстрационный стенд.

Инфраструктура для переезда подготовлена и проверена: собран и поднят весь стек, вход через nginx отработал. Переключение прода выполняется сменой DNS и требует решения владельца системы.

6.1.13.2 Схема

flowchart LR

```
user([Пользователь]) -->|HTTPS 443 / HTTP 80| nginx[Nginx]
nginx -->|/api/*| backend[FastAPI :8000]
```

```

nginx -->|/*| frontend[React / nginx :80]
backend --> pg[(PostgreSQL 16, volume)]
pg --> backup[/backups: pg_dump, 7 дней/]

```

Всё выполняется на одной VM Ubuntu 22.04/24.04 (2 vCPU, 4 ГБ RAM, 30 ГБ диска).

6.1.13.3 Два режима nginx Шаблон конфига выбирается при старте контейнера по `TLS_ENABLED`:

<code>TLS_ENABLED</code>	Что поднимается	Назначение
<code>false</code>	только 80, ответ по IP	отладка, пока нет домена и сертификата
<code>true</code>	80 (редирект) + 443 с сертификатом	демонстрация жюри

Разделение нужно потому, что с TLS nginx не стартует без файлов `/etc/letsencrypt/live/$DOMAIN/fullchain.pem` — без этого режима отладить стек по IP было бы невозможно.

6.1.13.4 Пошаговый план

1. Создать VM и статический публичный IP.
2. Выполнить `infra/yandex-cloud/setup-vm.sh` (Docker, certbot, ufw, swap, cron-бэкап, клонирование репозитория).
3. Заполнить `.env` по шаблону `infra/yandex-cloud/.env.prod.example`.
4. Поднять стек в режиме `TLS_ENABLED=false` и проверить `/api/health`.
5. Направить A-запись домена на IP, выпустить сертификат Let's Encrypt, переключить `TLS_ENABLED=true`.
6. Перенести данные из внешней БД: `pg_dump` → `pg_restore` в контейнер `rtk_postgres`.
7. Проверить `https://crm-rtk.pixel-minds.ru/api/health`, затем переключить DNS на IP VM.

Подробности и команды — в `infra/yandex-cloud/README.md`.

6.1.13.5 Что переносится автоматически, а что вручную

Автоматически (<code>docker compose</code>)	Вручную
Сборка и запуск backend, frontend, nginx	Создание VM и статического IP
Схема БД (создаётся приложением при старте)	Перенос данных (<code>pg_dump</code> / <code>pg_restore</code>)
Выбор конфига nginx по <code>TLS_ENABLED</code>	A-запись домена и выпуск сертификата
Ежедневные бэкапы (<code>cron</code> + <code>backup.sh</code>)	Заполнение секретов в <code>.env</code>

6.1.13.6 Смета Вычислительные ресурсы 748,80 ₽ + RAM 4 ГБ 950,40 ₽ + диск 30 ГБ 103,68 ₽ + публичный IP 189,73 ₽ = **1 992,61 ₽ в месяц**. Грант 5 000 ₽ покрывает около 2,5 месяцев непрерывной работы.

6.1.13.7 Обратная совместимость Базовый `docker-compose.yml` не изменялся, Yandex-вариант вынесен в отдельные файлы (`docker-compose.prod.yml` в корне — оверлей с локальным Postgres, `infra/yandex-cloud/docker-compose.prod.yml` — полный прод-стек с nginx). Локальный запуск через Vite-прокси тоже не затронут: `VITE_API_URL` по умолчанию пуст, поэтому запросы идут на origin страницы.

7 Технологический стек

7.1 Технологический стек

Документ фиксирует версии и назначение компонентов RTK CRM (кейс №6, ЛЦТ 2026).

7.1.1 Backend

Компонент	Версия	Назначение
Python	3.11	Язык и рантайм (образ python:3.11-slim)
FastAPI	0.141.1	HTTP API, валидация, OpenAPI
Uvicorn	0.34.0	ASGI-сервер
Pydantic / pydantic-settings	2.10.4 / 2.7.1	Схемы и конфигурация из окружения
SQLAlchemy (async)	2.0.36	ORM, async-движок
asyncpg	0.30.0	Драйвер PostgreSQL
aiosqlite	0.20.0	Драйвер SQLite для dev и тестов
Alembic	1.14.0	Миграции схемы
python-jose, passlib[bcrypt]	3.5.0 / 1.7.4	JWT и хеширование паролей
Redis client	5.2.1	Кэш отчётов, потенциально — счётчики лимитов
minio	7.2.10	S3-совместимое хранилище файлов
APScheduler	3.11.0	Фоновые проверки «зависших» заявок
prometheus-fastapi-instrumentator	8.1.0	Метрики на /metrics
openpyxl, xlwt, xlrd, reportlab	—	Экспорт отчётов XLSX/XLS/PDF
loguru	0.7.3	Логирование
pytest, pytest-asyncio	9.1.1 / 1.4.0	Тесты

7.1.2 Frontend

Компонент	Версия	Назначение
React	18.2	UI
TypeScript	5.2	Типизация
Vite	5.0	Сборка и dev-сервер
Ant Design	5.12	Компонентная библиотека
React Router	6.20	Маршрутизация
TanStack Query	5.x	Серверное состояние и кэш
Zustand	4.4	Клиентское состояние
Axios	1.6	HTTP-клиент
Recharts	3.x	Графики отчётов
dnd-kit	6.x / 8.x	Drag-and-drop этапов воронки

7.1.3 Инфраструктура

Компонент	Назначение
PostgreSQL	Основная БД (Yandex Managed PostgreSQL, контейнер на ВМ или Supabase)
Redis / KeyDB	Кэш агрегатов отчётов; KeyDB рекомендован заказчиком как более лёгкий
S3-хранилище	Файлы заявок: Yandex Object Storage или MinIO (S3-совместимое API)
Keycloak	Задел под корпоративную аутентификацию (закрытый контур)
Nginx	Reverse proxy, TLS, отдача статики
Docker Compose	Запуск всего контура одной командой

7.1.4 Внешние сервисы и интеграции

- **Yandex Cloud** — основной контур: Compute (ВМ), Managed PostgreSQL, Object Storage, Container Registry, Cloud DNS.
- **Render + Vercel + Supabase** — альтернативный контур без своей инфраструктуры (хостинг API, статика фронтенда, PostgreSQL).
- **GigaChat** — сводка по карточке взаимодействия (ФТ-6). Подключается по ключу авторизации; без него функция выключена и приложение работает как обычно.
- **LMS/Laravel** — задел под интеграцию (httpx), не в MVP.

7.1.5 Требования к окружению

- Python 3.11+, Node.js 20+.
- Для полного контура: Docker и Docker Compose.
- Минимум для запуска API без инфраструктуры: SQLite + локальный uploads/.

8 Аутентификация и авторизация

8.1 Аутентификация через Keycloak

В системе два режима аутентификации, переключаемых переменной AUTH_MODE:

Режим	Что проверяет	Когда используется
jwt (по умолчанию)	HS256-токен, подписанный SECRET_KEY	Локальный запуск, демонстрация, оба облачных контура
keycloak	RS256-токен Keycloak по JWKS	Закрытый контур заказчика, корпоративный SSO

Оба режима возвращают локальную запись User: она нужна для связей в БД (автор комментария, ответственный КАМ) и для RBAC. Различается только способ подтверждения личности.

8.1.1 Почему JWT остаётся режимом по умолчанию

В ТЗ Keycloak указан как вариант авторизации. Для прототипа JWT не требует отдельного сервиса, поэтому демонстрационный контур работает без Keycloak. Путь к корпоративному SSO при этом реализован полностью и включается одной переменной — переписывать код при требовании заказчика не нужно.

8.1.2 Состав

Файл	Назначение
keycloak/realm-export.json	Realm rtk-crm, клиент, роли, демо-пользователи
backend/auth/keycloak.py	Проверка токена по JWKS, разбор ролей
backend/auth/security.py	Выбор режима, RBAC по ролям токена
tests/test_keycloak_auth.py	20 тестов: подпись, claims, RBAC, кэш ключей

8.1.3 Структура realm

Realm rtk-crm содержит три роли и публичный клиент с PKCE:

- admin — администратор: справочники, пользователи, настройки;
- manager — менеджер: справочники и отчёты по всем вузам;
- kam — KAM: заявки и отчёты по своим вузам.

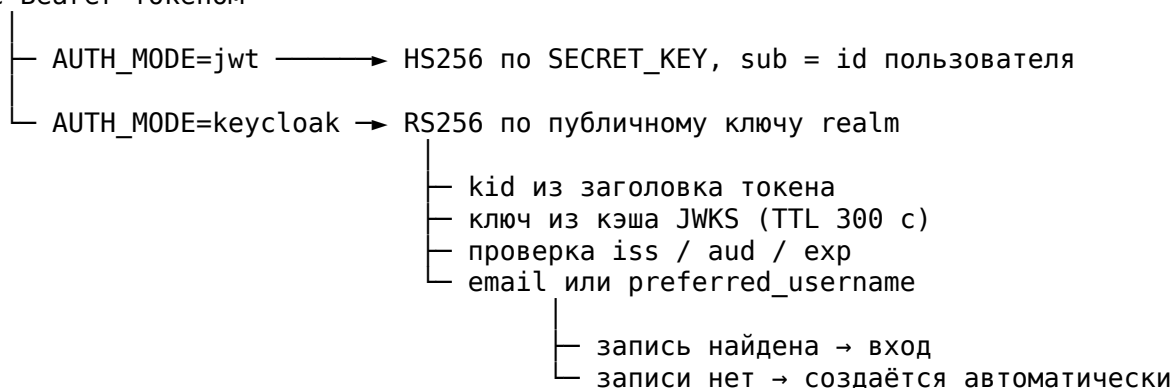
Клиент rtk-crm-frontend — публичный (без client secret), Authorization Code Flow с pkce.code.challenge.method=S256. Для публичного клиента PKCE обязателен: без него перехваченный код авторизации можно обменять на токен.

Роли кладутся в токен двумя мапперами: realm-роли (realm_access.roles) и audience (aud). Второй нужен, потому что по умолчанию Keycloak не проставляет aud клиента, а без него проверка audience не проходит.

Адреса возврата заданы шаблонами: http://localhost:3000/*, http://localhost:5173/* для локального запуска и http://* для стенда. Последний шаблон нужен, потому что адрес ВМ заранее неизвестен, а Keycloak принимает * только в конце строки: запись вида http://*/ не совпадает ни с одним адресом, и вход падает с Invalid parameter: redirect_uri.

8.1.4 Как работает проверка токена

Запрос с Bearer-токеном



Особенности реализации:

- **Кэш ключей.** Запрос к Keycloak делается не на каждый HTTP-запрос, а раз в KEYCLOAK_JWKS_TTL_SECONDS. Если в токене встретился неизвестный kid, кэш сбрасывается и ключи запрашиваются заново — так обрабатывается ротация.
- **Сбой Keycloak — это 503, а не 401.** Если JWKS недоступен, ответ 503 Service Unavailable: токен может быть корректным, и клиент не должен разлогиниваться из-за сетевой ошибки.
- **Роли из токена — источник истины.** В режиме keycloak RBAC опирается на realm_access.roles и resource_access.<client>.roles, а не на роль в БД. Смена роли в Keycloak действует сразу, без правки учётки в CRM.

- **Автосоздание учётки.** Если пользователь есть в Keycloak, но ещё не в CRM, запись создаётся при первом входе с ролью по приоритету admin > manager > kam. Локальный вход паролем для таких учёток невозможен: в hashed_password записан маркер !keycloak, который не является корректным bcrypt-хешем.

8.1.5 Запуск локально

```
export KEYCLOAK_ADMIN_PASSWORD=<пароль-админа-realm>
export DATABASE_URL=sqlite+aiosqlite:///./dev.db
export MINIO_ROOT_PASSWORD=<пароль-minio>
docker compose up -d keycloak
```

Проверка, что realm импортирован

```
curl -s http://localhost:8080/realms/rtk-crm/.well-known/openid-configuration | head
```

Realm импортируется из keycloak/realm-export.json при первом старте контейнера (--import-realm). Консоль администратора — http://localhost:8080.

Демо-пользователи из realm-файла:

Логин	Пароль	Роль
admin@rtk.ru	Admin123!	admin
manager@rtk.ru	Manager123!	manager
kam@rtk.ru	Kam123!	kam

Пароли в realm-файле предназначены для локальной демонстрации. Для закрытого контура их нужно заменить при импорте realm.

8.1.6 Включение в приложении

```
AUTH_MODE=keycloak
## Внутренний адрес: по нему backend забирает публичные ключи (JWKS).
KEYCLOAK_URL=https://keycloak.example.ru
## Публичный адрес: по нему браузер открывает Keycloak. Именно он попадает в
## claim `iss`, поэтому по нему проверяется issuer токена. В Docker это
## http://keycloak:8080 для KEYCLOAK_URL и внешний адрес VM для этого параметра.
KEYCLOAK_PUBLIC_URL=https://keycloak.example.ru
KEYCLOAK_REALM=rtk-crm
KEYCLOAK_CLIENT_ID=rtk-crm-frontend
## audience токена; пустое значение отключает проверку aud
KEYCLOAK_AUDIENCE=rtk-crm-frontend
KEYCLOAK_JWKS_TTL_SECONDS=300
```

Фронтенд переключается тем же режимом при сборке: VITE_AUTH_MODE=keycloak (в Docker Compose значение берётся из AUTH_MODE). В этом режиме вход выполняет Keycloak по Authorization Code Flow с PKCE, а access token подставляется в запросы и продлевается перед их отправкой. При VITE_AUTH_MODE=jwt остаётся локальная форма входа по email и паролю.

Адрес Keycloak для браузера задаётся через KEYCLOAK_PUBLIC_URL: из него собирается VITE_KEYCLOAK_URL, поэтому на VM достаточно указать KEYCLOAK_PUBLIC_URL=http://<ip-vm>:8080, и вход заработает без правки образа.

8.1.6.1 Страница входа В режиме keycloak на странице входа доступны обе опции: кнопка **Войти через Keycloak** и форма email/пароль под разделителем. Первая ведёт в единую систему входа, вторая остаётся рабочим запасным вариантом.

Перед инициализацией keycloak-js фронтенд проверяет доступность Keycloak по .well-known/openid-configuration. Это нужно потому, что при недоступном сервере keycloak-js в режиме check-sso уводит верхнее окно на Keycloak и оставляет пользователя на странице ошибки браузера. Предпроверка не даёт этому произойти: если сервер не ответил, страница сразу показывает форму входа.

8.1.7 Получение токена

Парольный грант (удобно для отладки API; в проде клиент использует Authorization Code + PKCE):

```
curl -s -X POST \
  http://localhost:8080/realms/rtk-crm/protocol/openid-connect/token \
  -d grant_type=password \
  -d client_id=rtk-crm-frontend \
  -d username=admin@rtk.ru \
  -d password='Admin123!' | python3 -m json.tool
```

Использование токена:

```
TOKEN=$(curl -s -X POST \
  http://localhost:8080/realms/rtk-crm/protocol/openid-connect/token \
  -d grant_type=password -d client_id=rtk-crm-frontend \
  -d username=admin@rtk.ru -d password='Admin123!' \
  | python3 -c 'import sys,json;print(json.load(sys.stdin)["access_token"])')
```

```
curl -s http://localhost:8000/api/auth/me -H "Authorization: Bearer $TOKEN"
```

8.1.8 Переход с JWT на Keycloak

1. Поднять Keycloak, импортировать realm, проверить JWKS.
2. Убедиться, что email пользователей CRM совпадает с email в Keycloak: сопоставление учётных записей идёт по этому полю.
3. Выставить AUTH_MODE=keycloak и перезапустить backend.
4. Проверить вход: GET /api/auth/me с токеном Keycloak.

Обратный переход — вернуть AUTH_MODE=jwt. Оба режима сосуществуют, данные пользователей не меняются.

8.1.9 Ограничения текущей реализации

- **Хранение состояния.** Локальный запуск использует start-dev с встроенной БД H2. Для закрытого контура нужен start --optimized с внешним PostgreSQL и включённым HTTPS: в dev-режиме Keycloak не рассчитан на нагрузку.
- **sslRequired: none** в realm-файле допускает работу по HTTP: демо-стенд открывается по http://<ip-vm>:8080, и при external Keycloak отклонял бы такой вход. Для промышленного контура требуется вернуть external и включить TLS на всех участках.
- **Выход из системы.** Logout выполняется на стороне Keycloak; backend не ведёт список отозванных токенов, поэтому access-токен живёт до exp. Уменьшение accessTokenLifespan в realm сокращает это окно.

9 Сводка по взаимодействию (GigaChat)

9.1 Сводка по взаимодействию через GigaChat (ФТ-6)

T3 допускает применение ИИ к данным о вузах. В RTK CRM это одна функция: сводка по карточке взаимодействия. КАМ открывает карточку и получает короткий текст — на каком этапе

вуз, что уже сделано, что требует внимания и какой следующий шаг. Цель практическая: восстановить контекст перед звонком, не перечитывая всю переписку.

9.1.1 Что попадает в сводку

Поле	Источник
Вуз	University.name
Продукт	ITProduct.name
Этап	WorkflowStageRef.name
Скоуп	Interaction.scope (B2B или B2C)
Ответственный КАМ	User.full_name
Задачи	Action.title, is_completed, due_date
Комментарии	Comment.text

Персональные данные в промпт не уходят. СНИЛС, паспортные данные, адреса и телефоны в карточке не хранятся — они отбрасываются ещё на этапе импорта (см. docs/SECURITY.md). В промпт передаются только названия, тексты комментариев и служебные поля.

9.1.2 Как это работает

Запрос идёт в два шага, как требует GigaChat:

1. Ключ авторизации (GIGACHAT_CREDENTIALS) отправляется в OAuth (/api/v2/oauth) в заголовке Authorization: Basic. В ответ приходит токен доступа на 30 минут. Заголовок RqUID обязателен — без него сервис отвечает 400.
2. Запрос к модели (/chat/completions) идёт с Authorization: Bearer.

Токен кэшируется в памяти процесса и обновляется заранее, за минуты до истечения срока. Один токен обслуживает все запросы, поэтому OAuth не вызывается на каждую сводку.

Ответ модели ограничен 150 словами и температурой 0.3: нужна сводка фактов из карточки, а не творческий текст. Системная инструкция прямо запрещает добавлять сведения, которых в данных нет.

9.1.3 Настройка

Минимально достаточно одного ключа:

GIGACHAT_CREDENTIALS=<authorization-key-base64>

9.1.3.1 Сертификат НУЦ Минцифры GigaChat предъявляет сертификат, выпущенный НУЦ Минцифры. Его корня нет в стандартном наборе доверенных сертификатов Python и большинства Linux-образов, поэтому без него TLS-соединение падает с CERTIFICATE_VERIFY_FAILED, и сводка не работает ни при каком ключе.

Сертификат поставляется в репозитории (backend/certs/russian_trusted_root_ca.pem), вшивается в образ (backend/Dockerfile кладёт его и в системный набор, и в /certs) и монтируется в контейнер из docker-compose.yml. Отдельно настраивать ничего не нужно: код сам берёт файл из /certs, а при локальном запуске — из backend/certs/. Переменная GIGACHAT_CA_BUNDLE нужна только если сертификат лежит в нестандартном месте.

Отключение проверки TLS (GIGACHAT_VERIFY_SSL=false) — крайняя мера для отладки: в рабочем контуре проверка включена.

9.1.4 Поведение без ключа и при сбоях

Функция спроектирована как необязательная: её отказ не должен ломать основной сценарий работы с карточкой. В демонстрационном контуре (`GIGACHAT_FALLBACK_ENABLED=true`) отказ внешнего сервиса не показывается пользователю как ошибка: вместо этого сводка собирается из данных карточки.

Ситуация	Ответ при <code>FALLBACK=false</code>	Ответ при <code>FALLBACK=true</code>
<code>GIGACHAT_CREDENTIALS</code> не задан	503	200, демо-сводка
OAuth отклонил ключ	502	200, демо-сводка
GigaChat недоступен или 5xx	502	200, демо-сводка
Неожиданный формат ответа	502	200, демо-сводка
Нет доступа к карточке	403	403

Демо-сводка — это детерминированный текст, собранный из полей карточки (`gigachat.build_fallback_summary`), вуз, продукт, этап, ответственный, счётчик задач и последний комментарий. Он не содержит сгенерированных утверждений, и в поле «Модель» такого ответа стоит демо-сводка (GigaChat не вызывался) — то есть демо-текст нельзя принять за ответ модели.

Ошибка внешнего сервиса не превращается в 500: по коду ответа видно, что проблема на стороне GigaChat, а не в CRM. Данные чужой карточки при этом во внешний сервис не уходят — доступ проверяется до сборки промпта.

9.1.5 Код

- `backend/services/gigachat.py` — авторизация, кэш токена, сборка промпта, разбор ответа, типы ошибок `GigaChatDisabled` и `GigaChatUnavailable`, демо-сводка `build_fallback_summary`. Модуль не вызывается из `/readyz`: недоступность необязательной функции не должна переводить сервис в `degraded`.
- `backend/api/interactions.py` — эндпоинт `POST /api/interactions/{id}/summary`.
- `frontend/src/components/interaction/InteractionDrawer.tsx` — вкладка «Сводка» с кнопкой и временем генерации.

9.1.6 Тесты

`tests/test_gigachat_summary.py` — 16 тестов. Внешний сервис подменяется транспортом `httpx`: GigaChat платный и недоступен в CI, а проверяется наш код, а не чужой сервис. Покрыты сборка промпта, оба шага авторизации, кэш токена, коды 503 и 502, запрет доступа к чужой карточке и оба режима демо-сводки — по отсутствию ключа и по сбою внешнего сервиса.

10 Безопасность и защита данных

10.1 Безопасность RTK CRM

Документ описывает реализованные меры защиты и соответствие требованиям 152-ФЗ, приказа ФСТЭК России № 117 и 149-ФЗ. Подробные результаты статического анализа и проверки зависимостей — в `security/SAST-SCA.md`.

10.1.1 Реализованные меры

- JWT Bearer-аутентификация с ограниченным временем жизни токена.
- RBAC: `user`, `manager`, `admin`; изменение справочников и `workflow` ограничено ролями.
- Пароли хранятся только как `bcrypt`-хеши.

- Запросы к базе выполняются через SQLAlchemy ORM.
- Изменяющие запросы фиксируются в `action_logs`; чувствительные поля (`password`, `token`, `secret`) редактируются перед записью.
- CORS использует явный список `origin`, без wildcard при `credentials`.
- Экспорт и файлы доступны только авторизованным пользователям.
- Загрузка файлов ограничена расширениями, MIME-типами и размером 50 МБ.
- Ограничение попыток входа: 5 неуспешных за 60 секунд с одного IP → 429.
- Заголовки безопасности на всех ответах: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy`, а в production — `Strict-Transport-Security`. Для API дополнительно `Content-Security-Policy: default-src 'none'; frame-ancestors 'none'; /docs и /redoc` из CSP исключены, потому что Swagger UI использует инлайновые скрипты.

10.1.2 152-ФЗ «О персональных данных»

Требование	Реализация
Правовое основание и согласие	Баннер cookie-согласия (<code>rtk_cookie_consent</code> в <code>localStorage</code>); обязательные и аналитические категории разделены
Минимизация данных	Хранятся рабочие данные пользователей (ФИО, email, роль) и контактных лиц вузов (ФИО, телефон, email)
Локализация БД	Целевой контур — Yandex Cloud (данные в РФ). Публичный демонстрационный стенд на внешнем хостинге для реальных ПДн не предназначен
Обезличивание	Экспорт и выгрузки содержат только рабочие идентификаторы и не раскрывают учётные данные; аудит-лог редактирует секреты
Сроки хранения	Определяются эксплуатантом; технически аудит-лог и <code>action_logs</code> поддерживают выборочную выгрузку и очистку
Ответственный за обработку	Назначается организацией-эксплуатантом

Обрабатываются рабочие данные пользователей и контактных лиц вузов. В промышленной эксплуатации необходимо разместить базу и backend в российском контуре (Yandex Cloud), ограничить доступ к персональным данным, определить сроки хранения и назначить ответственного за обработку. Публичный демонстрационный стенд на внешнем хостинге не следует использовать для реальных персональных данных без отдельной оценки и миграции.

Cookie-согласие хранится в `localStorage` под ключом `rtk_cookie_consent`; аналитические и маркетинговые категории по умолчанию отключены. Подробный план миграции в Yandex Cloud находится в `infra/yandex-cloud/README.md`.

10.1.2.1 Данные от кейсодержателя Кейсодержатель передал три файла с примерами выгрузок: `Вендоры.xlsx` (каталог ИТ-продуктов с контактами), `Данные оплат.json` (заявки B2C) и `Загрузка пользователей.xlsx` (анкета физлица).

`Загрузка пользователей.xlsx` — потенциально чувствительный файл: помимо ФИО, телефона и email он содержит колонки СНИЛС, серии и номера паспорта, адреса регистрации и данные диплома. **В переданном экземпляре эти колонки пусты** — заполнены только ФИО, телефон и email, то есть фактических персональных данных повышенной чувствительности файл не содержит.

Тем не менее исходные выгрузки как есть в репозиторий не попадают: любые таблицы со ПДн исключены правилами `.gitignore (*.xlsx, *.xls, Вендоры*, Данные оплат*, Загрузка пользователей*)`. Причины:

- анкета спроектирована под сбор СНИЛС и паспортных данных, и любой заполненный экземпляр нельзя коммитить;
- прототип не предназначен для хранения реальных паспортных данных — поля повышенной чувствительности в модели данных не реализованы;
- тестовые фикстуры для импорта собираются в памяти (`tests/test_excel_import.py`), поэтому исходные таблицы в репозитории не нужны.

Вместо исходных файлов в репозиторий положены **обезличенные образцы** тех же форматов (раздел «Обезличивание данных» ниже) — их достаточно для демонстрации импорта, и они не содержат ПДн.

В демо-данные (`backend/seed.py`) из этих файлов попадают только обезличенные сведения: справочник вендоров и продукты, а также заявки B2C без контактных данных. Телефоны и адреса почты заменены на синтетические значения в домене `example.ru`; реальные ФИО контактных лиц не сохраняются.

10.1.2.2 Обезличивание данных Образцы форматов выгрузок, нужные для демонстрации импорта, всё же добавлены в репозиторий — в обезличенном виде, каталог `docs/examples/` (зеркало — `frontend/public/docs/examples/`) и с латинскими именами, чтобы файлы корректно отдавались статикой на Linux:

Файл	Что обезличено
<code>vendors.xlsx</code>	Контакты вендоров — синтетические (<code>example.ru</code>)
<code>payments_b2c.json</code>	ФИО/телефон заявителей — тестовые
<code>users_b2c.xlsx</code>	ПДн-колонки заменены заглушками

В `users_b2c.xlsx` обезличены все поля персональных данных:

Поле	Значение
СНИЛС	000-000-000 00
Серия и номер паспорта	0000, 000000
Кем выдан паспорт	ОТДЕЛОМ УФМС РОССИИ
Дата выдачи паспорта	01.01.2020
Код подразделения	000-000
Адрес регистрации	г. Москва, ул. Примерная, д. 1, кв. 1
Индекс регистрации	000000

ФИО, телефон и email оставлены тестовыми — это вымышленные данные, не относящиеся к реальным лицам.

Пересборка обезличенной копии из оригинала: `python scripts/anonymize_case_data.py <путь к «Загрузка пользователей.xlsx»>`. Оригинал хранится вне репозитория.

Правила `.gitignore` по-прежнему исключают все выгрузки с ПДн (`*.xlsx, *.xls, Вендоры*, Данные оплат*, Загрузка пользователей*`); исключение сделано точно только для обезличенных образцов в `docs/examples/`.

Проверено: `git log --all --full-history -- "*Загрузка*"` — пусто. История репозитория не содержит реальных ПДн. Подробности — в `docs/examples/README.md`.

10.1.3 Приказ ФСТЭК России № 117

Мера	Реализация
Идентификация и аутентификация	JWT с bcrypt-хешами паролей; ограничение попыток входа
Управление доступом (RBAC)	Роли user/manager/admin, проверка через FastAPI-зависимости на каждом эндпоинте
Регистрация событий безопасности	Middleware audit_middleware → action_logs: пользователь, действие, сущность, IP, время
Антивирусная защита	На уровне инфраструктуры (документы к загрузке не исполняются; файлы отдаются как вложения)
Целостность	Ограничения БД, внешние ключи, миграции Alembic; уникальность этапов workflow
Доступ к журналам	Просмотр аудит-лога и выгрузка CSV ограничены ролью администратора
Шифрование канала	HTTPS/TLS на reverse proxy (Nginx); HSTS в проде
Резервное копирование	Ежедневный pg_dump в infra/yandex-cloud/backup.sh

10.1.4 149-ФЗ «Об информации...»

Регулирует оборот информации в системе: аудит-лог обеспечивает прослеживаемость изменений, ролевая модель — разграничение доступа к информационным ресурсам, а выгрузка данных — контролируемый формат (CSV/JSON/XLSX). Интеграция с внешними LMS/CMS выполняется по HTTPS с явными ключами доступа.

10.1.5 Секреты и эксплуатация

SECRET_KEY, DATABASE_URL и другие секреты передаются только через переменные окружения. В production приложение отклоняет дефолтный или слишком короткий SECRET_KEY. Файлы .env не должны попадать в Git.

10.1.6 GigaChat

Сводка по карточке — единственное обращение к внешнему ИИ-сервису, поэтому у него отдельные требования.

- Ключ авторизации (GIGACHAT_CREDENTIALS) хранится только в переменных окружения и в .env на сервере. В репозиторий он не попадает, в логах не печатается, в ответах API не возвращается.
- Ключ ротируется в личном кабинете GigaChat Developers. Компрометация ключа не даёт доступа к данным CRM: ключ используется только для запроса сводки.
- TLS-соединение проверяется по корневому сертификату НУЦ Минцифры. Он публичный, поставляется в backend/certs/ и вшивается в образ. Отключение проверки (GIGACHAT_VERIFY_SSL=false) не применяется в рабочем контуре.
- В промпт уходят только названия (вуз, продукт, этап), тексты комментариев и служебные поля. Персональные данные (СНИЛС, паспорт, адрес, телефон) в карточке не хранятся — они отбрасываются на этапе импорта, поэтому в GigaChat не попадают.
- Доступ проверяется до сборки промпта: КАМ не может получить сводку по чужому взаимодействию, данные чужой карточки во внешний сервис не уходят.
- Сервис работает в закрытом контуре Сбера и соответствует требованиям 152-ФЗ: персональные данные не передаются, обработка идёт в РФ.

Подробности: AI.md.

10.1.7 Аутентификация: JWT и Keycloak

В ТЗ Keycloak указан как вариант авторизации. Реализованы оба режима, переключаемые переменной `AUTH_MODE`:

`AUTH_MODE=jwt` (по умолчанию). Локальный контур для прототипа и демонстрации:

- `python-jose` подписывает Bearer-токены (HS256, ключ `SECRET_KEY`);
- `passlib` использует `bcrypt` для паролей;
- RBAC выполняется FastAPI-зависимостями;
- аудит сохраняет изменяющие операции без секретных полей.

`AUTH_MODE=keycloak`. Корпоративный SSO для закрытого контура:

- подпись токена проверяется по публичным ключам realm (RS256, JWKS);
- ключи кэшируются на 300 секунд, ротация обрабатывается по неизвестному `kid`;
- проверяются `iss`, `aud` и `exp`;
- роли берутся из токена (`realm_access` и `resource_access`), поэтому смена роли в Keycloak действует сразу;
- пользователь из Keycloak без учётки в CRM создаётся автоматически с ролью по приоритету `admin > manager > kam`; локальный вход паролем для него закрыт.

В обоих режимах возвращается локальная запись `User`: она нужна для связей в БД и RBAC, поэтому бизнес-логика не зависит от способа аутентификации.

Если Keycloak недоступен, API отвечает 503, а не 401: токен может быть корректным, и клиент не должен разлогиниваться из-за сетевой ошибки.

Подробности: `KEYCLOAK.md`.

Переменная `MOCK_MODE=true` фиксирует текущий режим JWT + БД. Автоматического переключения на Keycloak при `MOCK_MODE=false` нет — режим задаётся явно через `AUTH_MODE`, чтобы не включить SSO случайно при смене флага демо-данных.

11 Развёртывание

11.1 Деплой RTK CRM

Приложение не привязано к конкретному облаку: СУБД, объектное хранилище, кэш и список разрешённых `origin` задаются переменными окружения. Одна и та же сборка разворачивается в обоих поддерживаемых контурах.

Контур	Состав	Назначение
1. Yandex Cloud (основной)	BM Compute + nginx + PostgreSQL + Object Storage + KeyDB	Целевой контур: данные в РФ, 152-ФЗ, приказ ФСТЭК № 117
2. Render + Vercel + Supabase (альтернативный)	Render (backend), Vercel (frontend), Supabase (PostgreSQL)	Быстрый публичный стенд без своей инфраструктуры
3. Локально	Docker Compose или SQLite + Vite	Разработка и тесты

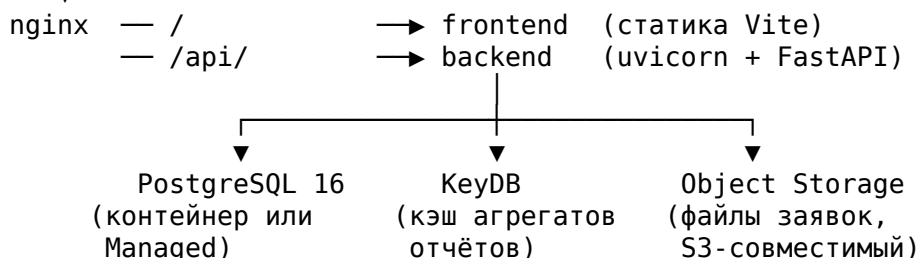
- 1. Yandex Cloud
- 2. Render + Vercel + Supabase
- 3. Локальный запуск
- 4. Общее для всех контуров

11.1.1.1 1. Yandex Cloud (основной контур)

Всё разворачивается на одной ВМ: PostgreSQL, KeyDB, backend (FastAPI), frontend (React/Vite) и nginx. Пошаговая инструкция по созданию ВМ — в `infra/yandex-cloud/README.md`.

Пользователь

↓ `https://crm-rtk.pixel-minds.ru` (или `http://<IP>` в режиме отладки)



Все контейнеры в одной docker-сети, наружу смотрят только nginx (80/443). Backend, база и кэш снаружи недоступны.

11.1.1.1.1 1.1. Рекомендуемые ресурсы ВМ

Параметр	Значение
vCPU	2 vCPU, 100 % уровня производительности
RAM	4 ГБ
Диск	30 ГБ
IP	статический (нужен для А-записи и сертификата)
Образ	Ubuntu 22.04 LTS

Замеры при сборке образов: пик памяти при `ci && tsc && vite build` — 1.69 ГБ, `pip install бэкенда` — 0.23 ГБ, рантайм всего стека — 143 МиБ в покое и ~210 МиБ под нагрузкой 50 пользователей. 4 ГБ хватает с запасом.

Доля vCPU важна отдельно: нагрузочный профиль из `tests/load/` упирается примерно в 1.6 ядра (backend ~105 %, PostgreSQL ~57 %). На 20 %-й доле этот профиль не покажет проектных цифр — для демонстрации переключайте на 100 % (меняется без пересоздания ВМ).

11.1.1.1.2 1.2. Установка на ВМ

на ВМ, после клонирования репозитория

```
cd infra/yandex-cloud
```

```
sudo ./setup-vm.sh
```

Скрипт ставит Docker, certbot, настраивает UFW и swap, копирует `.env.prod.example` в `.env`. Для приватного репозитория нужен `GITHUB_TOKEN` или `deploy-key`.

Далее заполните `.env` (шаблон — `infra/yandex-cloud/.env.prod.example`):

```
DOMAIN=rtk-crm.ru
```

```
TLS_ENABLED=true
```

Вариант А: Managed PostgreSQL

```
## DATABASE_URL=postgresql+asyncpg://rtk_user:<password>@c-<cluster-id>.rw.mdb.yandexcloud.net
```

Вариант Б: PostgreSQL в контейнере на ВМ

```
DATABASE_URL=postgresql+asyncpg://rtk_user:<password>@postgres:5432/rtk_crm
```

Кэш агрегатов отчётов

```
REDIS_URL=redis://keydb:6379/0
```

```
## Файлы заявок: Object Storage
S3_ENDPOINT=storage.yandexcloud.net
S3_SECURE=true
S3_REGION=ru-central1
S3_ACCESS_KEY=<yandex-access-key>
S3_SECRET_KEY=<yandex-secret-key>
S3_BUCKET=rtk-crm-files
```

```
SECRET_KEY=<уникальная строка не менее 32 символов>
CORS_ORIGINS=https://rtk-crm.ru
SEED_DEMO_DATA=true
```

```
## Сводка по карточке (ФТ-6). Без ключа функция выключена: 503 в API и
## пояснение во вкладке «Сводка». Сертификат НУЦ Минцифры вшит в образ,
## задавать GIGACHAT_CA_BUNDLE не нужно.
GIGACHAT_CREDENTIALS=<authorization-key-base64>
GIGACHAT_SCOPE=GIGACHAT_API_B2B
GIGACHAT_MODEL=GigaChat
GIGACHAT_FALLBACK_ENABLED=true
```

Имена переменных должны совпадать с теми, что читает backend/core/config.py (SECRET_KEY, SEED_DEMO_DATA, CORS_ORIGINS, S3_*). ENVIRONMENT=production включает проверку длины SECRET_KEY при старте — со слабым ключом приложение не поднимется.

Запуск:

```
docker compose -f infra/yandex-cloud/docker-compose.prod.yml --env-file .env up -d
```

11.1.1.3 1.3. Два режима nginx Режим выбирается переменной TLS_ENABLED:

TLS_ENABLED	Что поднимается	Когда нужен
false	только 80, отвечает по IP	отладка, домена и сертификата ещё нет
true	80 (редирект) + 443 с сертификатом	демонстрация жюри

В режиме true сертификат нужно выпустить **до** старта nginx: certbot --standalone занимает порт 80, поэтому одновременно с работающим nginx он не отработает. Сертификат можно получить и через Yandex Certificate Manager, если домен делегирован в Yandex Cloud DNS.

11.1.1.4 1.4. Managed-сервисы (опционально) Вместо контейнеров на ВМ можно подключить управляемые сервисы — код менять не нужно, достаточно переменных:

Сервис	Переменная	Значение
Yandex Managed PostgreSQL	DATABASE_URL	хост вида с- <id>.rw.mdb.yandexcloud.net:6432, ?ssl=require
Yandex Object Storage	S3_ENDPOINT, S3_SECURE, S3_REGION	storage.yandexcloud.net, true, ru-central1
Yandex Managed Redis	REDIS_URL	адрес кластера с TLS

Managed PostgreSQL требует NullPool (уже настроен в backend/db/session.py): внешний пул в режиме transaction pooling не должен удерживать соединения приложения.

11.1.1.5 1.5. Бэкапы infra/yandex-cloud/backup.sh кладётся в /etc/cron.daily/rtk-backup скриптом setup-vm.sh и делает pg_dump раз в сутки.

11.1.2 2. Render + Vercel + Supabase (альтернативный контур)

Контур без своей инфраструктуры: frontend — статика на Vercel, backend — контейнер на Render, база — Supabase PostgreSQL.

Vercel (frontend, статика Vite)
| https://<project>.vercel.app



Render (backend, Docker: uvicorn + FastAPI)
| https://<service>.onrender.com



Supabase PostgreSQL (Session Pooler, порт 5432)

Отличие от контура на BM: frontend и backend живут на разных доменах, поэтому запросы идут cross-origin и CORS_ORIGINS обязателен, а VITE_API_URL указывает на абсолютный адрес backend.

11.1.2.1 2.1. Supabase (база данных) Connection string (Session Pooler, порт 5432):

DATABASE_URL=postgresql+asyncpg://<user>:<password>@<host>.pooler.supabase.com:5432/postgres

Требования:

- префикс postgresql+asyncpg:// (не postgresql://);
- порт 5432 (Session Pooler — держит постоянное соединение, нужно asyncpg);
- SSL включён по умолчанию, дополнительных параметров не требуется.

Пароли в репозиторий не коммитятся. Значения — только в Render → Environment и локальном backend/.env.

11.1.2.2 2.2. Render (backend) Тип: Web Service (Docker).

В репозитории есть готовый Blueprint render.yaml: он задаёт сервис, регион, путь к Dockerfile и переменные окружения, поэтому в панели Render достаточно подключить репозиторий и заполнить секреты (sync: false). Если создавать сервис вручную, параметры те же:

Параметр	Значение
Repository	Pavel1778/rtk-crm
Branch	main
Environment	Docker
Dockerfile Path	backend/Dockerfile
Docker Context Directory	backend
Region	Frankfurt (EU Central) — ближе к Supabase eu-west-1
Instance Type	Free

Переменные окружения (Render → Environment):

Ключ	Значение
DATABASE_URL	значение из Supabase
SECRET_KEY	уникальная строка не менее 32 символов
ENVIRONMENT	production; включает HSTS и строгую проверку SECRET_KEY

Ключ	Значение
CORS_ORIGINS	https://<project>.vercel.app,http://localhost:5173
CORS_ORIGIN_REGEX	regex для хостов предпросмотра (необязательно)
SEED_DEMO_DATA	true
MOCK_MODE	true
GIGACHAT_CREDENTIALS	ключ авторизации из личного кабинета GigaChat Developers
GIGACHAT_SCOPE	GIGACHAT_API_B2B
GIGACHAT_MODEL	GigaChat
GIGACHAT_FALLBACK_ENABLED	true — демо-сводка вместо 502 при сбое сервиса

Сертификат НУЦ Минцифры вшит в образ (backend/Dockerfile) и попадает в /certs, поэтому на Render дополнительно настраивать TLS не нужно: GIGACHAT_CA_BUNDLE уже указывает на этот путь по умолчанию. Если GIGACHAT_CREDENTIALS не задан, вкладка «Сводка» в карточке сообщает, что сервис не настроен (ответ 503).

ENVIRONMENT должен быть ровно production (или prod). При другом значении HSTS не отдаётся, а проверка длины SECRET_KEY не применяется.

backend/start.sh — вспомогательный скрипт для деплоя без Docker (задаётся как Start Command); в образ он не встроен. Выполняет те же шаги: create_tables() → ensure_schema() → seed_reference().

11.1.2.3 2.3. Vercel (frontend)

Параметр	Значение
Framework Preset	Vite
Root Directory	frontend
Build Command	npm run build
Output Directory	dist

Ключ	Значение
VITE_API_URL	https://<service>.onrender.com

Frontend — SPA на React Router, поэтому нужен rewrite всех путей на index.html (иначе 404 на /reports, /directories, /settings). Файл frontend/vercel.json уже в репозитории:

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

11.1.2.4 2.4. Проверка контура

```
GET https://<service>.onrender.com/ -> {"service":"RTK CRM",...}
GET https://<service>.onrender.com/health -> {"status":"ok"}
GET https://<service>.onrender.com/api/health -> {"status":"ok","app":"RTK CRM","database":"ok"}
GET https://<service>.onrender.com/healthz -> {"status":"ok"}
GET https://<service>.onrender.com/readyz -> {"status":"ok","checks":{...}}
GET https://<service>.onrender.com/metrics -> метрики Prometheus
GET https://<service>.onrender.com/docs -> Swagger UI
```

11.1.2.5 2.5. Типичные ошибки

Симптом	Причина	Решение
Build: backend/ \$ / ошибка pre-deploy relation "users" does not exist	В Render заполнено Pre-Deploy Command Схема не создана	Очистить поле Проверить, что старт шёл через start.sh/CMD
connection timeout к Supabase CORS error в браузере	Порт 6543 (transaction pooler) с asyncpg CORS_ORIGINS без домена Vercel	Использовать 5432 (session pooler) Строка через запятую без пробелов
Invalid (interleaved) UTF-8 в логах	Нет PYTHONIOENCODING	Уже задано в Dockerfile
Медленные ответы	Render в US, Supabase в EU	Region Render: Frankfurt

11.1.3 3. Локальный запуск

11.1.3.1 3.1. Через Docker Compose

```
docker compose up -d --build
## frontend: http://localhost:5173
## backend: http://localhost:8000/docs
```

11.1.3.2 3.2. Без Docker

```
## Backend (SQLite, внешняя СУБД не нужна)
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r backend/requirements.txt
## .env в корне репозитория: DATABASE_URL=sqlite+aiosqlite:///./dev.db
python scripts/run_dev.py

## Frontend
cd frontend
npm install
npm run dev
```

11.1.4 4. Общее для всех контуров

11.1.4.1 4.1. Как это работает при старте

1. Dockerfile ставит зависимости из backend/requirements.txt и запускает uvicorn app.main:app --host 0.0.0.0 --port 8000 (код в /app, PYTHONPATH=/).
2. При старте (app.main:lifespan) создаётся схема БД (create_tables()) и **сверяется с моделями** (ensure_schema()): добавляет недостающие колонки и индексы, заменяет устаревшие глобальные UNIQUE на составные по scope), затем идемпотентно загружаются справочники (seed_reference()): 14 этапов B2B + 4 этапа B2C, пользователи, направления, продукты).
3. Если SEED_DEMO_DATA=true, при старте дополнительно заполняются демо-вузы и взаимодействия (только для пустой базы).

Сверка схемы нужна, потому что create_all(checkfirst=True) существующие таблицы не изменяет: база, созданная ранней версией приложения, молча оставалась

без scope, и загрузка демо-данных падала с column workflow_stages.scope does not exist. ensure_schema() доводит такую схему до моделей, не удаляя данные, и отмечает головную ревизию Alembic, чтобы последующий alembic upgrade head не пытался создать уже существующие таблицы.

11.1.4.2 4.2. Режимы аутентификации

Режим	MOCK_MODE	Что нужно	Использование
JWT + БД	true	Только БД и SECRET_KEY	Хакатонный контур, оба облака
Keycloak	false	Keycloak, realm, клиент и адаптер токенов	Закрытый контур заказчика

Сейчас API реализует JWT-аутентификацию через POST /api/auth/login. Значение false зарезервировано под Keycloak и не переключает backend автоматически.

11.1.4.3 4.3. Демо-доступ

Роль	Email	Пароль
Администратор	admin@rtk.ru	admin123
Руководитель	manager@rtk.ru	manager123
КАМ	kam@rtk.ru	kam123

11.1.4.4 4.4. Безопасность

- Пароли БД и SECRET_KEY — только в переменных окружения.
- backend/.env в .gitignore, в репозитории лишь .env.example.
- JWT-доступ ко всем эндпоинтам, кроме /health и логина.
- После публичного показа сменить пароль БД и отозвать использованные токены доступа.

12 Отчёт о верификации

12.1 Статическая верификация RTK CRM

Дата: 2026-09-22

Ветка: dev

Репозиторий: github.com/Pavel1778/rtk-crm

Проверены исходный код, конфигурация, документация, генерация экспортов, frontend build и разрешённый Locust-прогон. Браузерный smoke-test и Docker Compose не запускались.

12.1.1 Актуализация после performance/UI блока

Последующие коммиты на dev:

Коммит	Что сделано
4cac43e	Redis/in-process TTL-кэш отчётов, инвалидация после mutations, индексы и устранение N+1

Коммит	Что сделано
853de15	Отдельный сценарий ReportsUser для параллельных отчётов
0698e76	Документация JWT/Keycloak, MOCK_MODE, production URLs и ссылки проекта
354f67c	Vite manual chunks и перевод legacy UI-цветов на --atmr-* tokens
0df43eb	Исправлен выбор ReportsUser в Locust CLI (weight=1)

Фактический локальный прогон на SQLite backend:

- 50 пользователей / 60 секунд: 1613 запросов, 0 ошибок, aggregate p95 93 мс;
- 10 пользователей ReportsUser / 30 секунд: 322 запроса, 0 ошибок, p95 /api/reports 7 мс, aggregate p95 77 мс.
- 50 пользователей / 3 минуты, смешанный read/write: 4673 запроса, 0 ошибок, p95 create 32 мс, move 43 мс, comments 45 мс, upload 33 мс, XLSX 48 мс, XLS 76 мс, PDF 210 мс, aggregate p95 110 мс;
- 10 пользователей / 1 минута, exports-only: 308 запросов, 0 ошибок, p95 XLSX 37 мс, XLS 60 мс, PDF 180 мс;
- 10 администраторов / 30 секунд, toggle этапа: 308 запросов, 0 ошибок, p95 disable 17 мс, enable 16 мс.

Эти значения относятся к локальному SQLite окружению и не являются SLA production Render/Supabase.

12.1.2 Итог

- 🟢 OK — 35 пунктов;
- ⚠ Warning — 8 пунктов;
- 🐛 Bug — 0 незакрытых пунктов.

Найденные исправимые дефекты закрыты коммитами:

Коммит	Что исправлено
9611446	RBAC справочников и импортов, production SECRET_KEY, CORS origins, проверка MIME/расширений, аудит, имена экспортов, тест экспортов, системный шрифт PDF, удаление секрета из Compose
56d3fe0	корневой SECURITY.md, актуальная архитектура документации, responsive/theme tokens, легенда и размеры графиков
b69b813	фильтры отчёта по периоду, этапу, вузу, продукту, направлению и ответственному; фильтры экспортов
4155019	устранение hardcoded-цветов в отчётах
9643426	удаление credential defaults из deployment-файлов и секретов из deployment-документации
7324df1	React Query caching для страницы отчётов

12.1.3 Актуализация после этапов А и В (23.09)

Коммиты на dev:

Коммит	Что сделано
этап А	PDF-отчёт: динамические ширины колонок, перенос длинных значений, landscape при >5 колонках, DejaVu Sans; CORS с точным origin и expose_headers; единый date-picker диапазона с пресетами
этап А (доработка)	фильтр диапазона дат добавлен на Kanban-доску (date_from/date_to в /api/interactions/board), конец периода включается целиком, перевёрнутый диапазон даёт 400
этап В	адаптивная лупа поиска по вузу; проценты на слайсах диаграммы «Доля продуктов»; единый компонент таблицы этапов
7ccbbc0	пересортировка этапов: двухпроходное обновление порядка вместо прямого присваивания, устранён 500 из-за UNIQUE(scope, order)
017054c	единый адаптивный StageTable для конструктора workflow и админских настроек
d32a1f0	единый конфиг колонок отчёта для интерфейса и выгрузок PDF/XLSX/XLS/JSON

Проверки после этапов А и В:

- `python -m pytest — 48 тестов`, все зелёные (включая новые наборы `test_pdf_layout.py`, `test_cors.py`, `test_report_columns.py` и три регрессионных теста пересортировки этапов);
- `npx tsc --noEmit — 0 ошибок`; `npm run build — успешно`;
- браузерная проверка (Chromium, Playwright): таблица этапов на 375px показывает карточки, на 768px и 1440px — таблицу, горизонтального переполнения нет; страница отчётов на 375px использует короткие заголовки колонок (как в PDF), на 1440px — полные (как в XLSX);
- drag-and-drop этапов уходит одним запросом `POST /api/stages/reorder` и возвращает 200, порядок сохраняется в БД.

12.1.4 Актуализация после этапа С (23.09)

Коммиты на dev:

Коммит	Что сделано
3dde8ca	структурные проблемы валидации импорта и генератор XLSX-отчёта
de4002d	эндпоинт <code>POST /api/catalogs/import/report</code>
e0633d7	исправлена загрузка файлов из интерфейса (Content-Type для FormData)
896535b	сводка проверки и таблица проблем в окне импорта
d1bc614	кнопки «Импортировать только валидные / Отменить / Скачать отчёт»
a465903	тест смешанного импорта со счётчиком пропущенных

Коммит	Что сделано
d7b1077	сервис рендера руководств в PDF и сборка frontend/public/docs/*.pdf
579bdd2	вкладка «Помощь»: оглавление с якорями, поиск, скачивание PDF

12.1.4.1 С.1. Проверка корректности импорта Что проверено:

- `python -m pytest` — 60 тестов, все зелёные (добавлен `tests/test_import_report.py`: 8 модульных тестов валидации и 4 теста контракта эндпоинта — 401 без токена, XLSX с заголовками-счётчиками, 400 на неизвестный `catalog_type` и на не-Excel файл, импорт только валидных строк);
- `npx tsc --noEmit` — 0 ошибок; `npm run build` — успешно;
- браузерная проверка (Chromium, Playwright) на `/directories`: файл из 4 строк (1 ошибка, 2 предупреждения) открывает окно импорта, сводка показывает «Строк в файле: 4», «Ошибки: 1», «Предупреждения: 2», таблица проблем содержит строку, поле, причину и уровень; кнопки «Импортировать только валидные (3)», «Отменить» и «Скачать отчёт об ошибках (XLSX)» доступны. Импорт создаёт 2 записи, тост сообщает «Импортировано записей: 2; пропущено строк: 1», окно закрывается, горизонтального переполнения нет.

Важное исправление: глобальный заголовок `Content-Type: application/json` перекрывал `multipart`-границу, и импорт справочников через интерфейс отвечал `422 Field required: file`. Заголовок теперь снимается для `FormData`, поэтому сценарий импорта работает `end-to-end`.

12.1.4.2 С.2. Вкладка «Помощь»: навигация, поиск и PDF Что проверено:

- `python -m pytest` — 65 тестов, все зелёные (добавлен `tests/test_guide_pdf.py`: 5 тестов на конвертер — все разделы `USER_GUIDE.md` и `ADMIN_GUIDE.md` доезжают до PDF, кириллица и `inline` разметка не теряются, картинки встраиваются как `XObject`, отсутствующая картинка не ломает рендер);
- `npx tsc --noEmit` — 0 ошибок; `npm run build` — успешно;
- `python scripts/render_guide_pdfs.py` — собирает `user-guide.pdf` (4 страницы) и `admin-guide.pdf` (3 страницы) в `frontend/public/docs/`;
- браузерная проверка (Chromium, Playwright) на `/help`: оглавление содержит 10 разделов, поиск фильтрует список по заголовкам, клик по пункту оглавления прокручивает документ к нужному заголовку, переключение на руководство администратора перестраивает оглавление, кнопка «Скачать PDF» доступна, 5 картинок руководства отрисованы, горизонтального переполнения и ошибок в консоли нет.

Важное исправление: `id` заголовков начинались с цифры (`#3-dashbord-i-navigaciya`), из-за чего `querySelector` падал с `SyntaxError` и переход по оглавлению не работал. Якоря получили префикс `section-`. Заодно одиночная картинка `Markdown` больше не оборачивается в `<figure>` внутри `<p>` — устранено предупреждение `validateDOMNesting`.

12.1.5 1. Экспорты и отчёты

Проверка	Статус	Доказательство
XLSX генерируется через <code>openpyxl</code>	✅ OK	backend/services/excel_export.py:6-11,30-107
XLS генерируется через <code>xlwt</code> с именованным цветом	✅ OK	backend/services/excel_export.py:13-17,110-157
PDF генерируется <code>ReportLab</code>	✅ OK	backend/services/excel_export.py:19-27,160-193

Проверка	Статус	Доказательство
Кириллица в PDF	☐ OK	backend/services/excel_export.py:175-193; backend/Dockerfile:7-10
PDF landscape и повтор заголовка таблицы	☐ OK	backend/services/excel_export.py:166-173,235-248
Обязательные колонки экспорта	☐ OK	backend/services/excel_export.py:39-48,118-127,211-220
Content-Disposition и CORS expose headers	☐ OK	backend/api/reports.py:351-359,388-397,425-433; backend/main.py:63-70
ASCII-имена rtk-report-YYYYMMDD.ext	☐ OK	backend/api/reports.py:33-34
Bearer-защита /api/reports*	☐ OK	backend/api/reports.py:74-85,326-336,363-373,400-410; прямой запрос без токена ожидаемо даёт 401
In-memory экспорт	☐ OK	scripts/export_smoke.py; получены XLSX 5 KB, XLS 5 KB, PDF 47 KB; XLSX читается openpyxl, PDF имеет %PDF и DeJaVu

12.1.6 2. Данные графиков и фильтрация

Проверка	Статус	Доказательство
by_stage содержит все активные этапы, включая нулевые	☐ OK	backend/api/reports.py:140-166,192-200
by_product не теряет взаимодействия без продукта	☐ OK	backend/api/reports.py:168-190; категория Без продукта
Динамика заполняет пропущенные даты нулями	☐ OK	backend/api/reports.py:202-243
Фильтры периода/этапа/вуза/продукта/направления/ответственного	☐ OK	backend/api/reports.py:37-71,74-100; frontend/src/api/endpoints.ts:158-185
UI-фильтр периода и передача фильтра в экспорт	☐ OK	frontend/src/pages/ReportPage.tsx:32-62,117-152
Вертикальная диаграмма этапов и усечение подписей	☐ OK	frontend/src/pages/ReportPage.tsx:18-224
Полное имя этапа в tooltip	☐ OK	frontend/src/pages/ReportPage.tsx:21-216
Круговая диаграмма продуктов и responsive legend	☐ OK	frontend/src/pages/ReportPage.tsx:23-267
Динамика без точек и с minTickGap=24	☐ OK	frontend/src/pages/ReportPage.tsx:27-300
Loading/empty/error states	☐ OK	frontend/src/pages/ReportPage.tsx:63-86,170-176,235-237,273-275

12.1.7 3. Функциональное ТЗ 1-13

Пункт	Статус	Доказательство / примечание
1. Каталоги и импорт XLS/XLSX	OK	backend/api/catalogs.py:16-89; маппинг в backend/services/excel_import.py
2. Отчёты по периоду с фильтрами	OK	backend/api/reports.py:74-100; UI периода в frontend/src/pages/ReportPage.tsx:1152
3. Фильтры вуз/направление/продукт/ответственный	OK	API поддерживает все четыре фильтра: backend/api/reports.py:47-56
4. Recharts и экспорт	OK	frontend/src/pages/ReportPage.tsx:18300; export routes
5. Переходы этапов и комментарии	OK	backend/api/interactions.py:139-176, 328-345; frontend drawer/comments
6. PNG/JPEG/PDF/ZIP/GZIP/RAR/DOC/DOCX/XLS/XLSX	OK	backend/api/files.py:20-50, 74-98
7. Колонки XLS/XLSX/PDF	OK	backend/services/excel_export.py:39-48, 118-127, 211-220
8. JSON endpoint	OK	backend/api/reports.py JSON route c get_current_user
9. Создание/обновление workflow	OK	backend/api/stages.py:76-135; mutation доступна admin
10. Keycloak или MOCK_MODE	Warning	Текущий рабочий контур использует JWT+bcrypt: backend/auth/security.py:15-64; Keycloak присутствует только в legacy/target Compose
11. RBAC user/manager/admin	OK	backend/auth/security.py:67-88; справочники и каталоги защищены manager/admin
12. Web UI	Warning	Код и production build проверены, браузерный smoke-test запрещён текущим заданием
13. React Query caching	OK	frontend/src/main.tsx:13-21; отчёты используют useQuery: frontend/src/pages/ReportPage.tsx:3449

12.1.8 4. Безопасность и privacy

Проверка	Статус	Доказательство
JWT expiry 1440 минут	OK	backend/core/config.py:22-24
bcrypt	OK	backend/auth/security.py:15, 25-30
Production secret validation	OK	backend/core/config.py:39-48

Проверка	Статус	Доказательство
Явный CORS без wildcard при credentials	☐ OK	backend/main.py:59-71
User context в audit	☐ OK	backend/auth/security.py:42-64; backend/middleware/audit.py:33-65
Collection-level audit и редактирование секретных полей	☐ OK	backend/middleware/audit.py:77-121
MIME + extension + 50 MB upload limit	☐ OK	backend/api/files.py:20-50,74-94
SQLAlchemy ORM	☐ OK	Checked backend API/services; raw SQL only fixed health/DDI statements
.env не отслеживается, examples разрешены	☐ OK	.gitignore, tracked files backend/.env.example, frontend/.env.example
Старые секреты в Git history	⚠ Warning	Текущие файлы очищены. Проверка всех 952 blob-объектов не нашла реальных сторонних учётных данных (Supabase/Render/GigaChat): в истории только placeholder'ы. Ротация сторонних ключей не требуется. Уточнено в docs/CLEANUP_REPORT.md: если контур поднимался из docker-compose.yml без переопределения, заменить нужно только локальный SECRET_KEY

12.1.9 5. Workflow, RBAC и cookie consent

Проверка	Статус	Доказательство
Все этапы в конструкторе	☐ OK	frontend/src/pages/SettingsPage.tsx: загружает listStages(true), StageTable показывает этапы выбранной воронки
Только активные этапы в Kanban	☐ OK	frontend/src/pages/BoardPage.tsx и useStages('active')
Optimistic update/rollback	☐ OK	frontend/src/hooks/useStages.ts:27-59
409 при активных взаимодействиях	☐ OK	backend/api/stages.py:120-130
Impact и перенос при удалении	☐ OK	backend/api/stages.py:41-59,180-205;
Disabled opacity 0.55	☐ OK	frontend/src/components/workflow/DeleteWorkflowPage.tsx:401-404

Проверка	Статус	Доказательство
Cookie schema и necessary locked	☐ OK	frontend/src/lib/cookieConsent.ts:1-94; banner/settings components
Privacy routes и footer links	☐ OK	frontend/src/pages/CookiePolicyPage.PrivacyPolicyPage.tsx, frontend/src/components/AppFooter.tsx
Cookie-баннер на мобильных	☐ OK	scripts/capture_cookie_mobile.py — баннер 25-35% экрана, кнопки столбиком, модалка в пределах экрана

12.1.10 6. Нефункциональные требования и документация

Проверка	Статус	Доказательство / примечание
Production response targets	⚠ Warning	Локальный прогон достиг aggregate p95 93 мс, но production SLA отдельно не подтверждён
50 concurrent users	☐ OK	Реальный локальный прогон: 1613 запросов, 0 ошибок
10 concurrent report requests	☐ OK	Реальный ReportsUser: 322 запроса, 0 ошибок, /api/reports p95 7 мс
Write-профиль интерфейса	☐ OK	4673 запроса за 3 минуты: create/move/comment/upload/export без ошибок
Toggle workflow stage	☐ OK	308 запросов: disable/enable через PATCH без ошибок
SPA и viewport	☐ OK	frontend/index.html:5; React Router и Vite build
Коды 400/401/403/404/409/422/500	☐ OK	Проверены статически по FastAPI routes/handlers; 422 handler: backend/main.py:74-89
USER_GUIDE и ADMIN_GUIDE	☐ OK	Файлы присутствуют
SECURITY.md с 152-ФЗ и миграцией Yandex Cloud	☐ OK	docs/SECURITY.md:1-34
DEPLOYMENT.md	☐ OK	docs/DEPLOYMENT.md; credentials заменены placeholders
Render/Vercel/Supabase/Yandex Cloud	☐ OK	docs/architecture/ARCHITECTURE.md, c4-context.md, c4-components.md
ARCHITECTURE и C4 Mermaid	☐ OK	docs/pitch.md
Pitch 11 слайдов	☐ OK	docs/qa-jury.md
QA 20 вопросов/ответов	☐ OK	10 PNG в docs/images/, подставляются в руководства и PDF через scripts/render_guide_pdfs.py
Реальные screenshots в docs/images/	☐ OK	

12.1.11 7. Atomaro UI и качество кода

Проверка	Статус	Доказательство / примечание
--atmr-* tokens и theme classes	☐ OK	frontend/src/index.css:1-24; light class в MainLayout.tsx
Радиусы/spacing/responsive grids	☐ OK	frontend/src/index.css; report layout
Mobile table/export/chart handling	☐ OK	responsive classes in frontend/src/index.css; responsive legend in ReportPage.tsx
Footer safe-area и max-content	☐ OK	frontend/src/components/AppFooter.tsx 15
Hardcoded colors	☐ OK	Рабочие UI-использования переведены в --atmr-*; HEX остались только в определениях токенов и native color inputs
TypeScript build	☐ OK	source ~/.nvm/nvm.sh && npm run build; build завершён, Vite сообщил только размер bundle >500 KB
Python compileall	☐ OK	python3 -m compileall -q backend/... scripts tests/load
git diff --check	☐ OK	Выполнено после финальных правок
print/TODO/console.log/: any scan	☐ OK	Статический rg scan не выявил совпадений в проверенных директориях

12.1.12 Актуализация после аудита против ТЗ (23.09)

Коммиты на dev:

Коммит	Что сделано
f58b7a6	доступ к взаимодействиям по id: КАМ работает только со своими карточками
a8bf9c1	управление пользователями в интерфейсе (вкладка «Пользователи») и журнал аудита 152-ФЗ (GET /api/audit, раздел «Журнал»)
ef1069c	доступ к файлам наследует доступ взаимодействия; проверка общая в backend/api/access.py

12.1.12.1 Что проверено по бизнес-правилам ТЗ

- **«Вуз + продукт» единственным взаимодействием** — `_ensure_unique` проверяет и создание, и смену продукта; дубликат отклоняется 409.
- **Не более двух параллельных взаимодействий на вуз** — `_ensure_parallel_limit` применяется при создании и при возврате карточки в активные.
- **Скоуп воронки** — карточка В2С не встаёт в В2В-колонку и наоборот.
- **RBAC** — КАМ (user) ограничен своими взаимодействиями на чтении, обновлении, перемещении, удалении, комментариях и файлах; руководитель и администратор видят всё. Навигация и страницы админа скрыты для остальных ролей.

- **Назначение ответственного** — КАМ при создании становится ответственным сам; руководитель/администратор выбирают любого КАМ в окне создания и может передать существующую карточку.

12.1.12.2 Проверки

- `python -m pytest` — 90 тестов, все зелёные (`test_audit_log.py` — 8, `test_interaction_access.py` — 14, включая 5 сценариев доступа к файлам);
- `npx tsc --noEmit` — 0 ошибок; `npm run build` — успешно;
- `python scripts/render_guide_pdfs.py` — `admin-guide.pdf` пересобран с разделами про пользователей и журнал.

12.1.13 Актуализация после финального аудита (24.09)

Коммиты на dev:

Коммит	Что сделано
86d57b8	CI: линтер, типы и тесты в GitHub Actions, общий кодстайл
3527936	журнал аудита: фильтры по сотруднику и периоду, выгрузка CSV

12.1.13.1 Что добавлено/закрыто

- **Импорт JSON** — валидация верхнего уровня по JSON Schema (Draft 7): принимается массив объектов либо `{"data": [...]}`; ошибки возвращаются с путём до поля, пустые объекты записей и отсутствие обязательного Название отклоняются. Покрыто `tests/test_excel_import.py` (7 тестов).
- **Seed-скрипт** — `scripts/seed.py` (флаг `--reference`) как обёртка над `backend/seed.py`; общий бутстрап импорта вынесен в `scripts/_bootstrap.py`.
- **CORS** — хост стенда предпросмотра больше не зашит в код: шаблон приходит через `CORS_ORIGIN_REGEX`, по умолчанию выключен (см. `docs/DEPLOYMENT.md`).
- **SAST/SCA** — `docs/security/SAST-SCA.md`: пины зависимостей backend, `npm audit = 0` уязвимостей, остаточный риск `ecdsa 0.19.2` (PYSEC-2026-1325) принят и обоснован (используется HS256).
- **Архитектура** — `docs/architecture/functional.md` и `er-model.md` добавлены; `ARCHITECTURE.md` дополнен разделом масштабирования 300+ (KeyDB).

12.1.13.2 Проверки

- `python -m pytest` — 98 тестов, все зелёные;
- `ruff check backend/ tests/ scripts/ conftest.py` — All checks passed;
- `mypy backend/` — no issues found in 40 source files;
- `bandit -c pyproject.toml -r backend/` — 0 issues;
- `npx tsc --noEmit` — 0 ошибок; `npm run build` — успешно; `npm audit` — 0;
- `python scripts/seed.py` — справочники и демо-данные создаются идиоматично.

12.1.14 Актуализация после проверки структуры (24.09)

Проверка дерева проекта на расхождения выявила два дефекта; оба закрыты.

12.1.14.1 1. Зеркало документации расходилось с источниками Вкладка «Помощь» отдаёт Markdown из `frontend/public/docs/`, но эта копия поддерживалась вручную и отстала от `docs/`:

- ADMIN_GUIDE.md, SECURITY.md, ARCHITECTURE.md содержали более ранние редакции (не было воронки B2C, кэша отчётов, rate limit, планировщика);
- architecture/functional.md, er-model.md, security/SAST-SCA.md и STACK.md вообще отсутствовали, хотя исходные документы на них ссылаются, — внутренние ссылки в Help вели в никуда;
- картинка 10-help-docs.png отличалась от исходной.

Теперь зеркало собирается детерминированно: scripts/sync_public_docs.py копирует набор документов и переписывает относительные ссылки в абсолютные (/docs/...), потому что страница рендерится по маршруту /help. Повторный прогон не меняет дерево. За дрейфом следит tests/test_docs_mirror.py (6 тестов): синхронизация воспроизводима, все ссылки разрешаются, относительных ссылок и битых картинок нет.

12.1.14.2 2. Бэкенд в контейнере не находил конфиг колонок отчёта backend/services/report_ читал конфиг как Path(__file__).parents[2] / "config" / "report_columns.json". В образе код лежит в /app (build context — backend/), поэтому путь указывал на /config/report_columns.json, а каталог config/ в образ не попадал — выгрузки XLSX/XLS/PDF и /api/reports/columns падали бы в Docker.

Исправлено: путь выбирается из кандидатов (корень репозитория и /config) с поддержкой явного REPORT_COLUMNS_CONFIG; в оба compose-файла добавлен read-only mount ./config:/config. Покрыто тестами в tests/test_report_columns.py.

12.1.14.3 Проверки

- python -m pytest — 110 тестов, все зелёные (было 98);
- ruff check backend/ tests/ scripts/ conftest.py — All checks passed;
- mypy backend/ — no issues found in 40 source files;
- npx tsc --noEmit — 0 ошибок; npm run build — успешно;
- python scripts/sync_public_docs.py — идиомпотентен, зеркало совпадает с docs/;
- bash backend/start.sh — поднимается и отдаёт {"status":"ok"} на /api/health.

12.1.15 Актуализация: документация запуска (24.09, второй проход)

Проверка команд из README и docs/DEPLOYMENT.md на исполняемость выявила, что документированный локальный запуск backend не работает.

12.1.15.1 3. uvicorn backend.main:app не мог запуститься Приложение импортируется как пакет app (в контейнере — /app, в репозитории — backend/), при этом backend/main.py импортирует app.*. Поэтому uvicorn backend.main:app из корня падал с ModuleNotFoundError: app. Именно поэтому тесты и scripts/_bootstrap.py создают временный алиас app -> backend.

Добавлен scripts/run_dev.py — делает пакет импортируемым и поднимает uvicorn с автоперезагрузкой (проверено: /api/health отвечает 200). README и docs/DEPLOYMENT.md переведены на него.

12.1.15.2 4. backend/start.sh использовал тот же нерабочий путь Render-скрипт импортировал backend.db.session / backend.seed и запускал uvicorn backend.main:app. Теперь он создаёт алиас app -> backend (как в conftest.py) и работает через app.*; проверено запуском.

12.1.15.3 5. Расхождения в документации

- docs/README.md: дерево проекта описывало несуществующий вид (backend/api/models/, backend/api/api/, отсутствие auth/, middleware/, services/, config/); технологическая

таблица указывала FastAPI 0.104 (фактически 0.141) и активный Keycloak 24.0 (фактически JWT); команда инициализации БД была `docker exec rtk_backend python seed.py` — в контейнере код в `/app`, поэтому верно `python /app/seed.py` или `python scripts/seed.py` из репозитория.

- README.md и docs/DEPLOYMENT.md: `.env` предлагалось класть в `backend/`, но `Settings` читает `.env` из текущего каталога — при запуске из корня файл игнорировался.
- README.md: ссылка на docs/ARCHITECTURE.md, которого нет (docs/README.md и docs/architecture/ вместо него).
- docs/DEPLOYMENT.md: утверждалось, что `start.sh` — это CMD образа; на деле CMD — `uvicorn app.main:app`, схема и справочники создаются в `lifespan` приложения.
- docs/architecture/ARCHITECTURE.md: Keycloak 24.0 был указан как активный Auth Server; заменено на JWT + `bcrypt` с пометкой про целевой контур.

За регрессиями следит `tests/test_dev_entrypoint.py`.

12.1.16 Критично: падение деплоя на Render (24.09)

Прод падал при старте с `FileNotFoundError: '/config/report_columns.json'` и `Exited with status 1`. Разбор:

- `backend/services/report_columns.py` читает конфиг **на импорте модуля**;
- Docker/Render собирают образ с `build context backend/`, поэтому корневой `config/` в образ не попадает;
- в Compose путь закрывался монтированием `./config:/config:ro`, а на Render монтирования нет — `/config/report_columns.json` отсутствует, импорт `app.main` падает, контейнер завершается с кодом 1.

Исправление:

- канонический конфиг дублируется в `backend/config/report_columns.json` (`scripts/sync_report_c` копия попадает в образ);
- `report_columns.config_candidates()` проверяет пути по порядку: `bundled backend/config/` → `корневой config/` → `/config`;
- `REPORT_COLUMNS_CONFIG` по-прежнему переопределяет путь.

Проверено симуляцией `layout` образа (`app/` = только содержимое `backend/`, без корневого `config/`): импорт `app.main` и `PDF_COLUMNS` проходят. Регрессии — `tests/test_report_columns.py` (`bundled`-копия, порядок кандидатов, идемпотентность синхронизации).

Дополнительно исправлено:

- `scripts/test_export.py` → `scripts/export_smoke.py`: скрипт назывался `test_*` (pytest его не собирает, т.к. лежит вне `tests/`) и падал с `ModuleNotFoundError: app` — импортировал `backend.services.*` без `bootstrap`-алиаса. Теперь использует `ensure_app_importable()` и запускается.

12.1.17 Осталось

1. Ротировать `credentials`, которые ранее попали в историю Git; удаление из текущего состояния не отзывает старые значения.
2. Отдельно проверить `production latency` на Render/Supabase; локальный `target` достигнут.
3. При необходимости заменить JWT на Keycloak/добавить явно документированный `MOCK_MODE`, если это обязательное условие комиссии, а не альтернативный вариант.

12.1.18 Прод-инцидент 24.09: схема БД без scope (исправлено)

Симптом (лог Render): Ошибка загрузки демо-данных: `column workflow_stages.scope does not exist`; прод-база пустая; GET / → 404.

Причина: прод-схема создана ранней версией приложения через `create_all(checkfirst=True)`, который существующие таблицы не изменяет. Alembic-миграция схему описывала, но не применялась: `alembic.ini` лежит в корне репозитория, а Docker build context — `backend/`, поэтому CLI в образе недоступен. В прод-БД отсутствовали `workflow_stages.scope`, `interactions.scope`, а уникальность задавалась глобально по `code/order`.

Исправление (коммит 5cf3855, PR #13):

Изменение	Что делает
<code>backend/db/schema_sync.py</code>	идемпотентная сверка схемы с моделями: добавляет <code>scope</code> (DEFAULT 'b2b') и индексы, заменяет глобальные UNIQUE на составные (<code>scope, code</code>)/(<code>scope, order</code>); SQLite — пересоздание с копированием данных, Postgres — ALTER TABLE; помечает головную ревизию Alembic
<code>backend/main.py</code>	<code>ensure_schema()</code> в <code>lifespan</code> ; GET / отдаёт ссылки вместо 404
<code>backend/start.sh</code> <code>README.md</code> , <code>docs/DEPLOYMENT.md</code> , <code>LoginPage.tsx</code>	тот же порядок инициализации без Docker реальный демо-аккаунт KAM kam@rtk.ru/kam123 вместо несуществовавшего user@rtk.ru

Проверка после деплоя (24.09, Render/Supabase):

- GET / → 200 {"service": "RTK CRM", ...};
- GET /healthz → 200; GET /readyz → 200 (database: ok);
- этапы: 18 (14 B2B + 4 B2C);
- вузы: 5; взаимодействия: 7 — демо-данные загружены.

Локальная верификация: 125 тестов зелёные, ruff/муру чисто, tsc без ошибок; E2E на legacy-схеме — сверка применена, `compare_metadata` без расхождений.

12.1.19 Актуализация: запуск одной командой и CI SourceCraft (26.09)

Задача — чтобы `docker compose up -d` работал на чистой машине и чтобы проверки выполнялись и на зеркале в SourceCraft.

12.1.19.1 1. docker compose не собирался без переменных окружения `docker compose config` завершался с ошибкой интерполяции: `compose` требует `MINIO_ROOT_PASSWORD` и `KEYCLOAK_ADMIN_PASSWORD` (синтаксис `${VAR:?}`), а шаблона `.env.example` в корне репозитория не было — существовали только `backend/.env.example` и `frontend/.env.example`, где этих имён нет.

Добавлен корневой `.env.example`. Имена переменных взяты из `docker-compose.yml` и `backend/core/config.py`, а не из стороннего описания: в частности, хранилище настраивается через `MINIO_ROOT_*/S3_*`, а не `MINIO_ACCESS_KEY/MINIO_SECRET_KEY`, ключ GigaChat называется `GIGACHAT_CREDENTIALS`, а не `GIGACHAT_AUTH_KEY`.

Проверено: `docker compose --env-file <тест> config → exit 0`, 7 сервисов (redis, minio, minio-init, backend, frontend, keycloak, nginx).

12.1.19.2 2. Переменные не доходили до контейнера backend `Compose` передавал в `backend` только `DATABASE_URL`, `REDIS_URL`, `S3_*`, `KEYCLOAK_URL`, `PYTHONPATH` и `CORS_ORIGINS`. Всё остальное, включая `SECRET_KEY`, `AUTH_MODE` и `GIGACHAT_*`, в контейнер не попадало: приложение работало на значениях по умолчанию из `config.py`, а переключить `AUTH_MODE=keycloak` через `.env` было невозможно.

Добавлена передача SECRET_KEY, AUTH_MODE, KEYCLOAK_REALM, KEYCLOAK_CLIENT_ID, KEYCLOAK_AUDIENCE, GIGACHAT_*, SEED_DEMO_DATA и параметров уведомлений. Логин администратора Keycloak вынесен в KEYCLOAK_ADMIN_USERNAME (раньше был захардкожен admin).

Проверено разбором docker compose config: значения доходят до сервиса backend.

12.1.19.3 3. Миграции и seed вручную запускать не нужно backend/main.py в lifespan выполняет create_tables() → ensure_schema() → seed_demo(). Поэтому команда вида alembic upgrade head && python -m scripts.seed && uvicorn ... в контейнере нерабочая: alembic.ini лежит в корне репозитория и в образ не попадает (build context — backend/), а scripts/ в образ не копируется. Это уже приводило к прод-инциденту 24.09 (см. раздел выше). В README зафиксировано, что инициализация автоматическая.

12.1.19.4 4. CI для SourceCraft Добавлен .sourcecraft/ci.yaml с теми же проверками, что в .github/workflows/ci.yml: ruff, муру, pytest для backend и tsc --noEmit + npm run build для фронтенда. Пути соответствуют структуре репозитория — тесты лежат в tests/, а не в backend/tests/.

12.1.19.5 Проверка контейнеров Ранее запуск контейнеров выполнить не удавалось: Docker-демон был недоступен, доступна только CLI, поэтому проверка была статической. Docker поднят, стек проверен вживую.

Что запускалось: redis, backend, frontend, nginx — собраны из репозитория и подняты в одной пользовательской сети. Образы minio/minio и minio/mc в этой среде недоступны (pull access denied), поэтому S3-хранилище проверялось в режиме S3_ENDPOINT="": файлы пишутся в локальный uploads/. Контейнер keycloak проверялся отдельно при отладке redirect_uri.

Проверка	Результат
GET /healthz	200, {"status": "ok"}
GET /readyz	200,
GET /	200, отдаёт ссылки на docs/healthz/readyz
Старт приложения	схема сверена (ensure_schema), демо-данные загружены, планировщик запущен
POST /api/auth/login (admin, manager)	токен выдан
GET /api/auth/me с токеном	200, роль и is_admin соответствуют пользователю
GET /api/universities без токена	401
GET / через frontend (nginx)	200, отдаётся index.html приложения
SPA-fallback (/interactions)	200
/api/auth/me через прокси frontend	200 — same-origin схема работает
/api/health, /health через корневой nginx	200

Найден и исправлен дефект корневого nginx: конфиг nginx/nginx.conf отдавал location / из /usr/share/nginx/html, но в образе nginx собранной статика нет — запрос на порт 80 возвращал дефолтную страницу «Welcome to nginx!», хотя README указывает http://localhost как точку входа. Теперь location / проксирует на контейнер frontend, как в прод-контуре на BM (infra/yandex-cloud/nginx-http.conf.template).

readyz со статусом unavailable по кэшу воспроизводится только при запуске контейнеров в дефолтной сети Docker: имена сервисов там не разрешаются, и backend не видит redis. В пользовательской сети проверка проходит. На BM используется отдельная сеть, поэтому к продукту это не относится.

12.1.20 Актуализация: ER-модель, вёрстка презентации и Q&A (27.09)

Финальный проход перед сдачей. Три направления проверены отдельно, каждое — на реальном артефакте, а не только по коду.

12.1.20.1 1. ER-диаграмма стала воспроизводимой Раньше схема данных жила только текстом в `er-model.md`. Теперь Mermaid-блок извлекается скриптом `scripts/extract_er_diagram.py` в `docs/architecture/er.mmd`, а `mmdc` рендерит его в `er-model.pdf` и `er-model.png`.

Дополнительно собран файл Archi `docs/architecture/er.archimate` (ArchiMate 3, 5.0.0): 32 элемента в папке «Данные (ER)» — 11 сущностей, 14 связей и 7 Mini-Of-Entity. Файл открывается в Archi бесплатно, рядом лежит готовый PDF для тех, кому Archi не нужен.

Проверка	Результат
<code>scripts/extract_er_diagram.py</code>	пишет <code>er.mmd</code> , 107 строк, начинается с <code>erDiagram</code>
Рендер <code>mmdc -i er.mmd -o er-model.pdf</code> <code>er.archimate</code>	PDF ~170 КБ, одна страница well-formed XML, <code>id="model-rtk-crm-er"</code> , 32 элемента
Ссылки в <code>ARCHITECTURE.md</code> и <code>architecture/README.md</code>	добавлены, зеркало синхронно

12.1.20.2 2. Вёрстка презентации проверена рендером Добавлен `scripts/check_presentation.py`. Он конвертирует `.pptx` в PDF через LibreOffice и ищет два класса дефектов: текст за границей слайда и наложение текстовых блоков. Эвристики по `python-pptx` не используются — они давали ложные срабатывания на групповых фигурах шаблона.

Проверено при 960×540 pt: 11 слайдов, 0 дефектов. Метаданные чистые: `author` и `last_modified_by` — «Сабадаш Павел».

12.1.20.3 3. Вопросы жюри разложены на два блока `docs/qa-jury.md` переработан: часть I — 20 продуктовых вопросов (проблема, пользователь, Kanban, 14 этапов, доступ, данные, 152-ФЗ, экспорт, нагрузка, масштабирование, мобильные, следующий шаг), часть II — 20 технических (интеграция, версионирование workflow, аудит, RBAC, целостность при параллельном обновлении, XLSX-импорт, KeyDB, LLM, Docker, документация). Каждый ответ ссылается на конкретный файл, а не пересказывает ТЗ.

12.1.20.4 Вывод Все три артефакта воспроизводятся одной командой, зеркало документации не расходится с источниками (`tests/test_docs_mirror.py` — 6 passed), а у презентации нет визуальных дефектов при рендере.

12.1.21 Актуализация: cookie-баннер на мобильных (27.09)

Проверка на реальных размерах экранов выявила пять дефектов в согласии на cookie, которые видны только на телефоне.

12.1.21.1 Что было сломано

Дефект	Проявление
Баннер занимал экран <code>column + flex-wrap</code>	На iPhone SE (375×667) баннер — 70% высоты, кнопки столбиком в три ряда Кнопки уезжали во вторую колонку за пределы экрана

Дефект	Проявление
z-index баннера 1100	Перекрывал футер модальки настроек:
Модалка выше экрана	«Сохранить выбор» не нажималась
Хардкод отступа	4 категории + текст не влезали, кнопки оказывались за нижней кромкой
	padding-bottom: 168px был меньше реальной высоты мобильного баннера

12.1.21.2 Что изменено

- `.cookie-banner`: z-index снижен до 900 (модалка `antd` — 1000, поэтому футер снова доступен); ограничение `max-height: 55vh` со скроллом внутри; `env(safe-area-inset-bottom)` для iPhone с «чёлкой».
- Мобильные `.cookie-banner__actions`: `flex-wrap: nowrap`, столбик, кнопки на всю ширину — три ряда подписей больше не обрезаются.
- Модалка `.cookie-settings-modal`: на $\leq 575\text{px}$ ограничена `max-height: calc(100vh - 24px)`, тело скроллится, футер — `column-reverse` с кнопками на всю ширину.
- `CookieConsentBanner.tsx`: высота баннера измеряется `ResizeObserver` и пишется в `--cookie-banner-h`; `body.has-cookie-banner` берёт отступ из этой переменной вместо хард-кода.
- Политики `.policy-card`: текст 14px, таблица cookie-категорий скроллится локально (`scroll={{ x: 640 }}`), заголовки сохранены крупнее.

12.1.21.3 Проверка Скрипт `scripts/capture_cookie_mobile.py` снимает до/после и проверяет баннер, кнопки, модальку, `localStorage` и страницы политик. Прогон на старой сборке воспроизводит дефекты, на новой — проходит:

before: `iphone-se`: баннер занимает 70% экрана

before: `iphone-se/galaxy-s20/iphone-14-pro-max`: кнопку сохранения модальки перекрывает другой эл

after: OK

Устройство	Высота баннера до	после
iPhone SE 375×667	70%	35%
Galaxy S20 360×800	—	29%
iPhone 14 Pro Max 430×932	—	25%
iPad 820×1180	—	12%
Desktop 1440×900	—	9%

Дополнительно: ни на одной странице нет горизонтальной прокрутки, отступ `body` совпадает с высотой баннера на всех проверенных маршрутах, модалька и её футер целиком в пределах экрана, контраст кнопок 13.9 (dark) и 16.8 (light). Полный прогон backend-тестов — 200 passed.

Скриншоты: `docs/images/mobile/cookie-{before,after}-*`, `policy-after-*`. Зеркало документации синхронизировано (`python scripts/sync_public_docs.py`).

12.1.22 Синхронизация репозитория (27.09)

Работа идёт в двух зеркалах: `github.com/Pavel1778/rtk-crm` — основное, `git.sourcecraft.dev/lct-hackaton-2026/case-16-edtech-crm-team-54` — зеркало, с которого рабочая VM на Yandex Cloud тянет код для демо-стенда.

Рабочий прототип: <https://crm-rtk.pixel-minds.ru> (домен, HTTPS, Let's Encrypt).

Состояние на момент отчёта:

Площадка	Ветка	Примечание
GitHub	main	источник, сюда попадают все коммиты
SourceCraft	main	зеркало, с него ВМ тянет код для демо-стенда

Порядок синхронизации — GitHub → SourceCraft: сначала коммит уходит в основное зеркало, затем та же история переносится в SourceCraft и оттуда подтягивается на ВМ. Токен SourceCraft в репозиторий и в `.git/config` не пишется: используется `GIT_ASKPASS` или `credential helper`, чтобы секрет не осел в рабочей копии.

12.1.23 Обезличивание данных кейсодержателя

Кейсодержатель передал три выгрузки: `Вендоры.xlsx` (каталог ИТ-продуктов и вендоров), `Данные оплат.json` (заявки В2С) и `Загрузка пользователей.xlsx` (анкета физлица). В анкете есть поля персональных данных: СНИЛС, серия и номер паспорта, кем и когда выдан, код подразделения, адрес регистрации и индекс.

Перед добавлением в репозиторий анкета была **специально обезличена**: все ПДн-поля заменены на сгенерированные заглушки (000-000-000 00, 0000 000000, ОТДЕЛОМ УФМС РОССИИ, 01.01.2020, 000-000, г. Москва, ул. Примерная, д. 1, кв. 1, 000000). ФИО, телефон и email оставлены тестовыми. Пересборка — `scripts/anonymize_case_data.py`. Оригинал хранится вне репозитория. Это соответствует требованиям 152-ФЗ и политике безопасности проекта (см. `docs/SECURITY.md`, `docs/examples/README.md`).

Обезличенные образцы лежат в `docs/examples/` вместе с зеркалом в `frontend/public/docs/examples/`: `vendors.xlsx`, `payments_b2c.json`, `users_b2c.xlsx`. Именование латиницей — чтобы файлы не ломались на Linux-стенде и в URL вкладки «Помощь».

Проверено: `git log --all --full-history -- "*Загрузка*"` — пусто, реальные ПДн в истории отсутствуют.

12.1.23.1 Что входит в сдачу

- `docs/qa-jury.md` — 40 вопросов (20 продуктовых и 20 технических) с опорой на файлы кода, а не на пересказ ТЗ.
- `docs/architecture/er.mmd`, `er-model.pdf`, `er.archimate` — ER-модель: Mermaid-исходник, отрендеренная диаграмма и модель ArchiMate 3 (32 элемента).
- `docs/rtk-crm-documentation.pdf` — единый PDF (63 страницы) со всеми разделами, включая ER и Q&A для жюри.
- `docs/presentation/RTK-CRM-LCT2026.pptx` — презентация, 11 слайдов.
- `docs/examples/` — образцы форматов от кейсодержателя для демонстрации импорта: `vendors.xlsx`, `payments_b2c.json`, `users_b2c.xlsx` (обезличен). Синхронизируются в зеркало.
- `frontend/public/docs/` — зеркало документации для вкладки «Помощь» (`scripts/sync_public_docs` ссылки переписаны в абсолютные `/docs/...`).

12.1.23.2 Осталось до сдачи

- **Деплой и проверка стенда** <https://crm-rtk.pixel-minds.ru> — Postgres, Redis/KeyDB, MinIO, Keycloak, backend, frontend, nginx.
- **MinIO на стенде.** В изолированной среде сборки образы MinIO не тянулись, поэтому прототип проверялся на `S3-fallback` с локальным `uploads/`. На ВМ MinIO поднимается (`pgsty/minio`) — нужно убедиться, что backend пишет вложения именно туда, а не на локальный диск.
- **Нагрузочный отчёт на целевом контуре.** Локальный прогон Locust дал 1613 запросов за 60 секунд, 0 ошибок, aggregate p95 93 мс. Цифры на ВМ нужно перепроверить и обновить `tests/load/report.html`.

13 Коды ошибок

13.1 Каталог кодов ошибок API

Требование НФТ-3 ТЗ: у ошибок должны быть стабильные коды, чтобы фронтенд показывал понятное сообщение по коду, а не только по HTTP-статусу.

Формат ответа при ошибке:

```
{
  "detail": "Человекочитаемое сообщение",
  "code": "MACHINE_READABLE_CODE"
}
```

Код	HTTP	Когда возникает	Пример сообщения
AUTH_INVALID_CREDENTIALS	401	Неверный email или пароль	«Неверный email или пароль»
AUTH_TOKEN_EXPIRED	401	JWT истёк или некорректен	«Сессия истекла, войдите снова»
FORBIDDEN_ROLE	403	Роли не хватает прав	«Недостаточно прав»
ACCOUNT_DISABLED	403	Учётная запись отключена	«Учётная запись отключена»
RESOURCE_NOT_FOUND	404	Сущность не найдена	«Вуз не найден»
STATUS_HAS_ACTIVE_REQUESTS	409	Удаление этапа с заявками	«На этапе N взаимодействий. Укажите перенос»
DUPLICATE_INTERACTION	409	Дублирующая пара вуза и продукта	«Такое взаимодействие уже зарегистрировано»
DUPLICATE_VUZ_NAME	409	Дубль вуза при импорте	«Вуз уже существует»
MAX_PARALLEL_INTERACTIONS	409	Больше 2 активных взаимодействий у вуза	«У вуза уже 2 активных взаимодействия»
WORKFLOW_SCOPE_MISMATCH	409	Этап из другого workflow	«Этап принадлежит другому workflow»
VALIDATION_FAILED	422	Ошибка Pydantic-схемы	«Поле X обязательно»
IMPORT_FILE_CORRUPTED	422	Битая кодировка или структура файла	«Файл повреждён, проверьте кодировку»
RATE_LIMITED	429	Превышен лимит попыток входа	«Слишком много попыток, попробуйте через минуту»
INTERNAL_ERROR	500	Непредвиденная ошибка	«Внутренняя ошибка, попробуйте позже»

13.1.1 Реализация

- MAX_PARALLEL_INTERACTIONS и WORKFLOW_SCOPE_MISMATCH возвращаются в backend/api/interactions
- RATE_LIMITED — в backend/middleware/rate_limit.py.
- VALIDATION_FAILED — глобальный обработчик RequestValidationError в backend/main.py.

14 Руководство пользователя

14.1 Руководство пользователя RTK CRM

14.1.1 1. Назначение системы

RTK CRM помогает КАМ и руководителям вести взаимодействие с вузами: от первого контакта до контроля результата. В системе есть Kanban-доска, карточки взаимодействий, задачи, комментарии, файлы и отчёты.

14.1.2 2. Вход в систему

Способ входа зависит от контура, в котором работает система.

Единая система входа (Keycloak). Нажмите **Войти через Keycloak**. Браузер переходит на страницу корпоративного входа, где вы вводите свои учётные данные. Пароль вводится на стороне Keycloak и в приложение не попадает. После успешного входа вы возвращаетесь в RTK CRM уже аутентифицированными.

Вход по учётной записи CRM. Если единая система входа недоступна или контур работает в локальном режиме, используйте форму ниже разделителя: введите рабочую электронную почту и пароль, нажмите **Войти**.

Если учётная запись заблокирована или пароль неизвестен, обратитесь к администратору. Пароли не передаются в чатах и не записываются в документы.

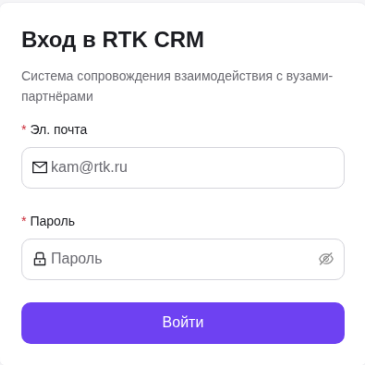


Рис. 1: Экран входа

14.1.3 3. Дашборд и навигация

После входа открывается Kanban-доска. В верхнем меню доступны:

- **Доска** — текущие взаимодействия по активным этапам;
- **Отчёты** — показатели, графики и экспорт;
- **Справочники** — вузы, продукты и направления;
- **Помощь** — эта документация внутри приложения.

Менеджеру дополнительно видна вкладка **Интеграция**, а администратору — разделы **Пользователи**, **Журнал** и **Интеграция**. Настройка этапов workflow живёт внутри раздела **Пользователи** на вкладке «Этапы воркфлоу» и описана в руководстве администратора.

Светлая, тёмная тема и режим «как в системе» переключаются кнопкой справа в шапке.

На мобильном экране меню открывается кнопкой с тремя полосами.

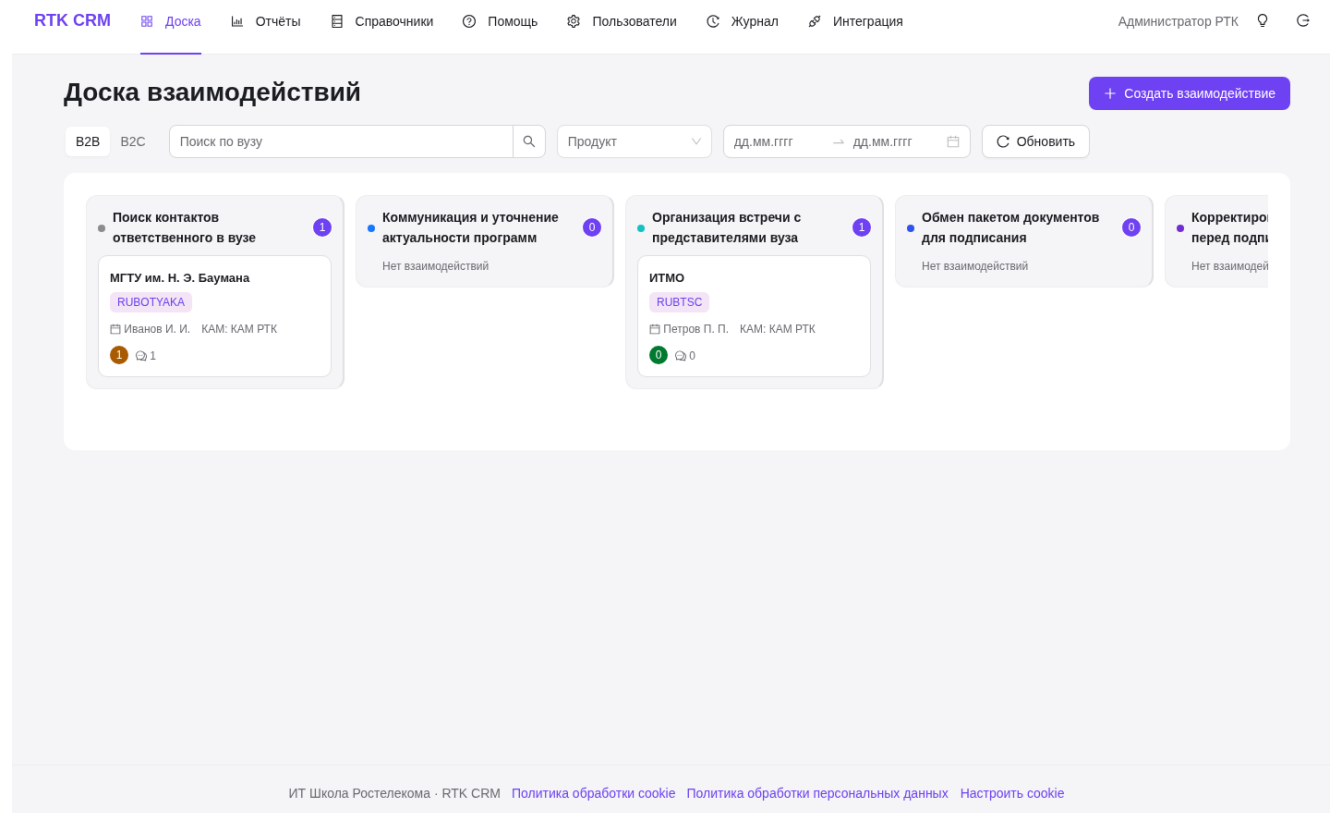


Рис. 2: Доска взаимодействий

14.1.4 4. Kanban-доска

Каждая колонка — активный этап процесса. Чтобы переместить взаимодействие, перетащите карточку в другую колонку. На телефоне сначала выберите нужный этап в фильтре, затем откройте карточку.

Карточка показывает вуз, продукт, ответственного и счётчики задач, комментариев и файлов. Неактивные этапы не отображаются на доске, но данные взаимодействий сохраняются.

Над доской доступны фильтры: поиск по названию вуза, продукт, диапазон дат и выбор этапа на телефоне. Диапазон дат отбирает взаимодействия по дате создания: выберите начало и конец периода или готовый пресет («Сегодня», «7 дней», «30 дней», «Этот месяц», «Прошлый месяц», «Квартал», «Год»). Конец периода включается целиком, поэтому взаимодействия, созданные вечером последнего дня, остаются в выборке.

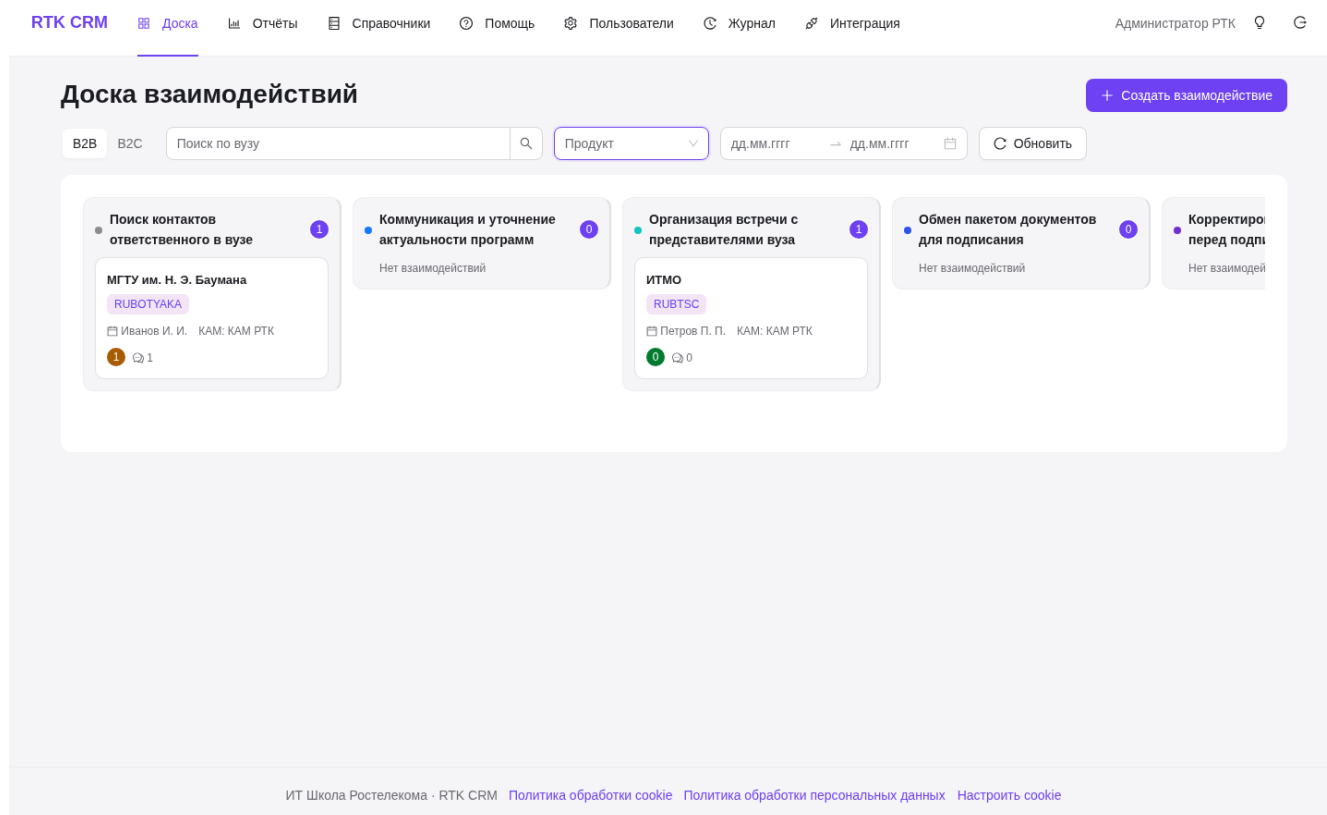


Рис. 3: Фильтры на доске

14.1.5 5. Карточка взаимодействия

Откройте карточку кликом. В ней доступны:

- основные сведения о вузе, продукте и договоре;
- задачи с датами и статусом выполнения;
- комментарии и история изменений;
- прикрепленные файлы.

Изменения сохраняются кнопкой действия в соответствующем разделе.

14.1.6 6. Создание взаимодействия

1. Откройте доску.
2. Нажмите **Добавить взаимодействие**.
3. Выберите вуз, продукт и начальный этап.
4. Сохраните форму.

Ограничение: у одного вуза может быть не более двух активных взаимодействий. Если лимит достигнут, система покажет понятное сообщение об ошибке.

14.1.7 7. Отчёты и экспорт

На странице **Отчёты** доступны KPI, распределение по этапам, распределение по продуктам и динамика взаимодействий за 30 дней.

Переключатель **B2B / B2C** задаёт воронку: у каждой свой набор этапов, и отчёт показывает только выбранную. Тот же выбор влияет на таблицу предпросмотра и на все загрузки.

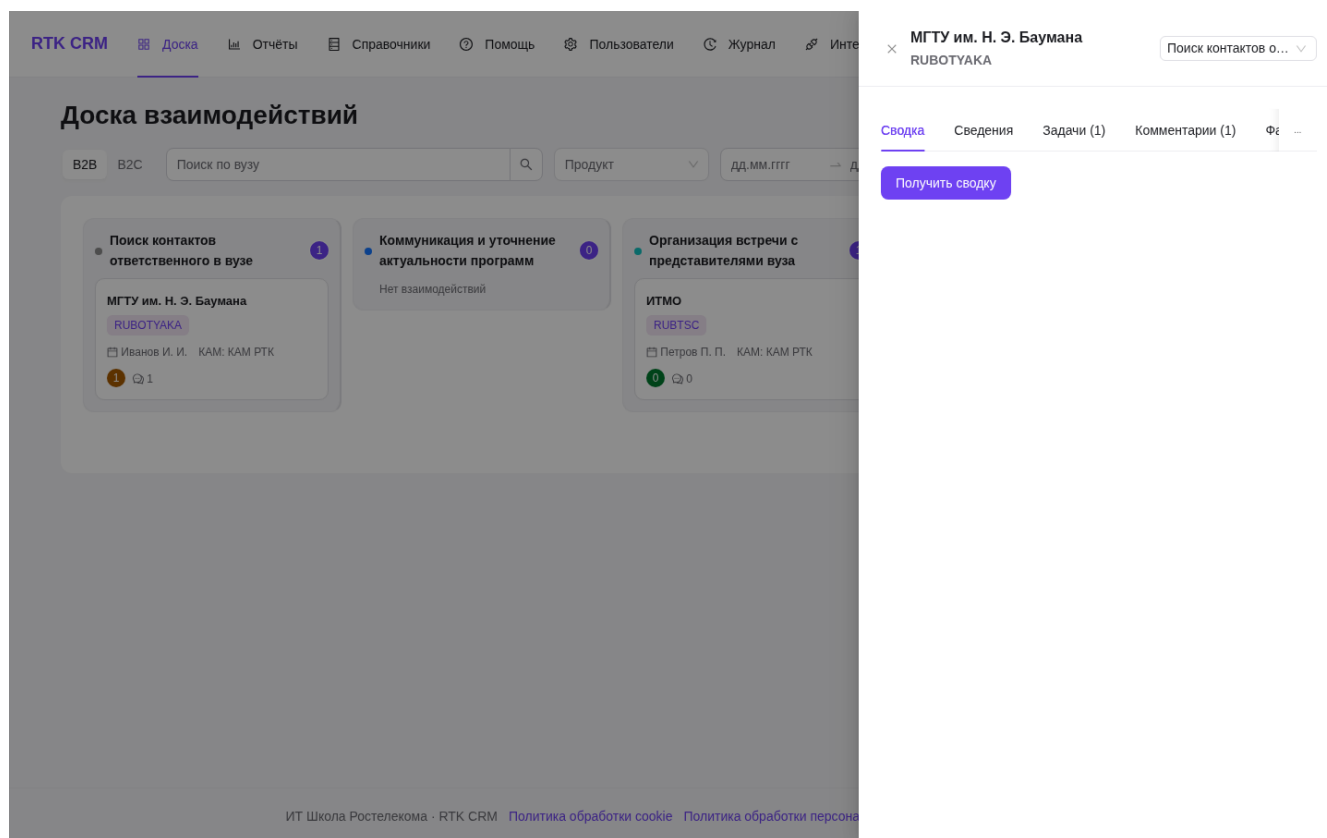


Рис. 4: Карточка взаимодействия

Нажмите **Экспорт XLSX**, **Экспорт XLS** или **Экспорт PDF**. Файл скачивается через авторизованный запрос.

14.1.8 8. ИИ-сводка по взаимодействию

В карточке взаимодействия есть кнопка **ИИ-сводка**. Она собирает накопленную историю — этап, задачи, комментарии — и возвращает короткое резюме: на каком этапе взаимодействие, что уже сделано и что осталось. Это экономит время при передаче карточки коллеге и при подготовке к встрече.

Сводку формирует сервис GigaChat. Если он не настроен, поведение зависит от контура:

- в демонстрационном режиме возвращается сводка, собранная из полей карточки, с явной пометкой, что модель не вызывалась;
- в рабочем режиме без ключа функция отвечает ошибкой с кодом 503 — это означает, что сервис недоступен, а не что данных нет.

Повторное нажатие перестраивает сводку по текущему состоянию карточки.

14.1.9 9. Интеграция с LMS и CMS

Раздел **Интеграция** связывает CRM с внешними системами заказчика: учебной платформой (LMS) и сайтом (CMS). Обмен идёт пакетами JSON по контракту — описанной схеме полей.

Входящий поток (импорт из LMS). Вставьте пакет в поле ввода или нажмите **Загрузить пример**, затем **Предпросмотр**. Система проверит пакет по контракту и покажет, что будет импортировано, без записи в базу. Если проверка прошла, нажмите **Импортировать**.

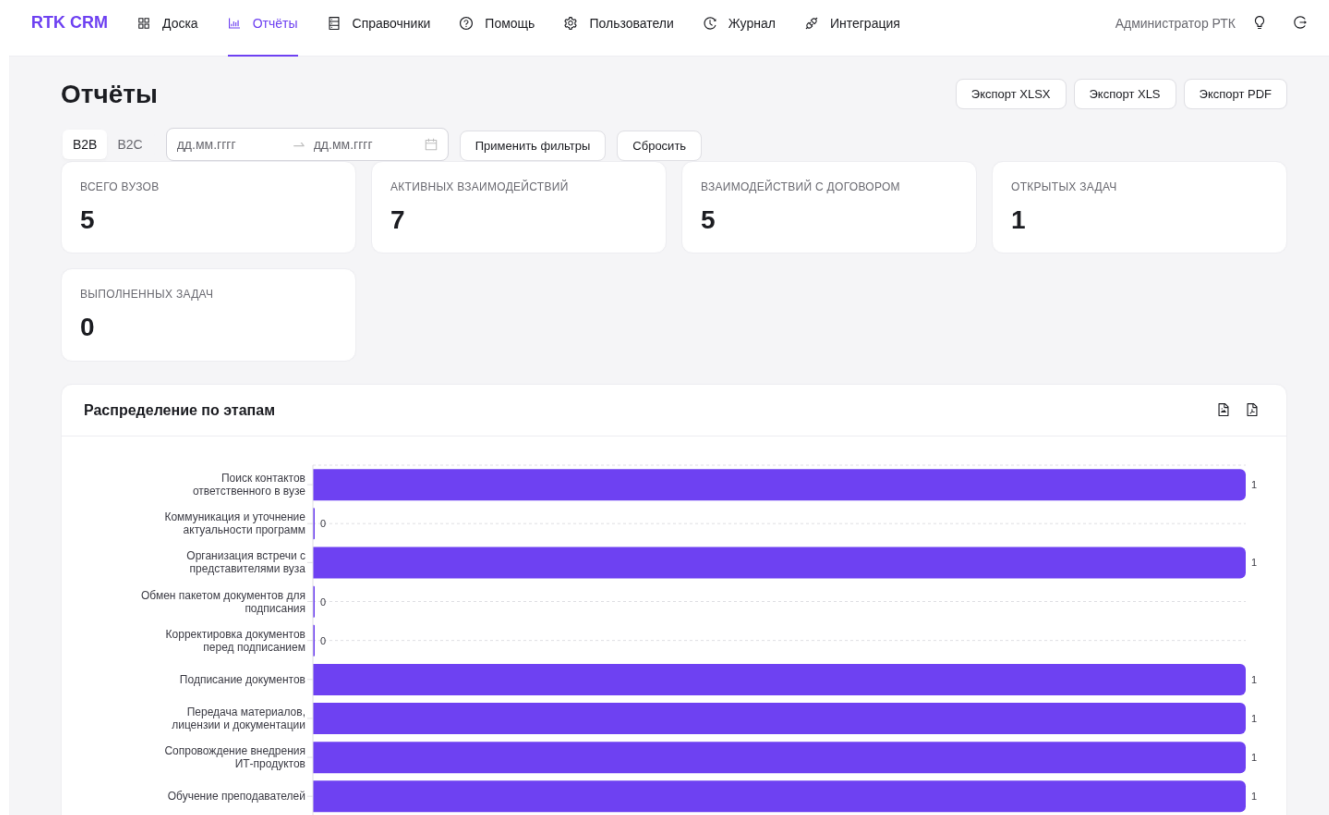


Рис. 5: Страница отчётов

Идемпотентность. Импорт опирается на `contract_number` — номер договора. Повторная загрузка того же пакета обновляет существующую запись, а не создаёт дубль. Поэтому повторный запуск обмена безопасен.

Исходящий поток (выгрузка в LMS). Кнопка **Выгрузить пакет** формирует набор записей для передачи во внешнюю систему в том же формате контракта.

Схема обмена. На странице показана структура контракта: какие поля обязательны и как они называются. Это описание служит основой для интеграции с реальными системами — прототип использует моск-адаптер, который отдаёт детерминированные данные по тому же контракту.

14.1.10 10. Cookie и приватность

При первом посещении появляется баннер согласия. Можно принять все категории, отклонить необязательные или открыть **Настроить**. Необходимые cookie всегда включены. Изменить выбор можно в футере через **Настроить cookie**.

Политики доступны по адресам `/cookie-policy` и `/privacy`.

14.1.11 11. Частые вопросы

Почему этапа нет на доске? Администратор мог отключить его. Проверьте настройки workflow.

Почему экспорт вернул ошибку? Проверьте, что вы вошли в систему, затем повторите скачивание. При повторной ошибке сообщите администратору время и формат файла.

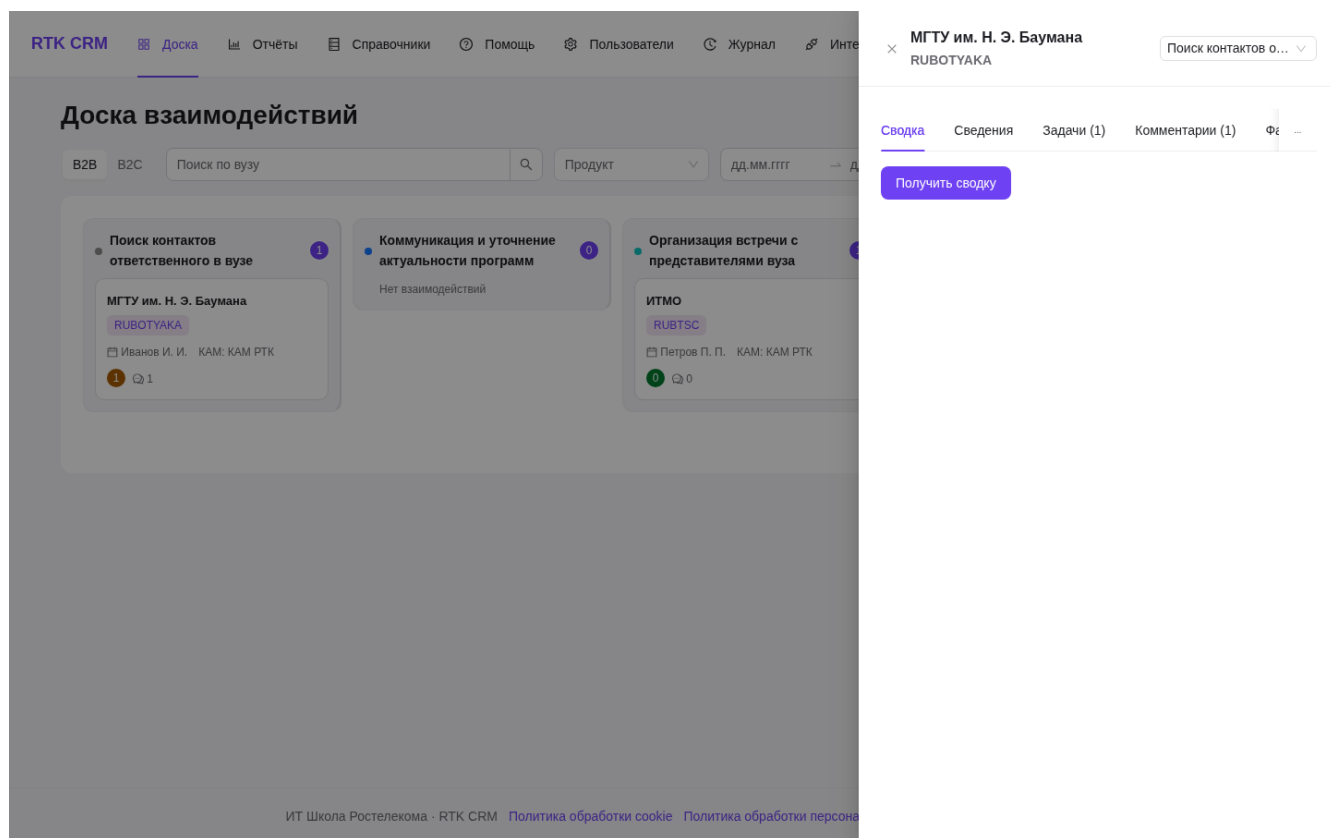


Рис. 6: Карточка взаимодействия

Как получить доступ к справочникам? Доступ зависит от роли. Обратитесь к администратору, если нужный раздел скрыт.

14.1.12 12. Справка внутри приложения

Раздел **Помощь** содержит это руководство, руководство администратора, описание мер безопасности и архитектуры. Слева — поиск по разделам и список заголовков: выбранный раздел подсвечивается при прокрутке, клик переносит к нему. Для руководств пользователя и администратора кнопкой **Скачать PDF** доступна версия для печати.

Иллюстрации в разделах выше сняты на демонстрационных данных; реальные персональные данные в них отсутствуют.

15 Руководство администратора

15.1 Руководство администратора RTK CRM

15.1.1 1. Роль администратора

Администратор отвечает за пользователей, справочники, этапы workflow и доступность системы. Операционные действия выполняются после проверки прав и бизнес-ограничений.

15.1.2 2. Пользователи

Управление пользователями находится в разделе **Пользователи** (вкладка «Пользователи»). Для каждого пользователя задаются email, имя, роль и активность. При деактивации доступ

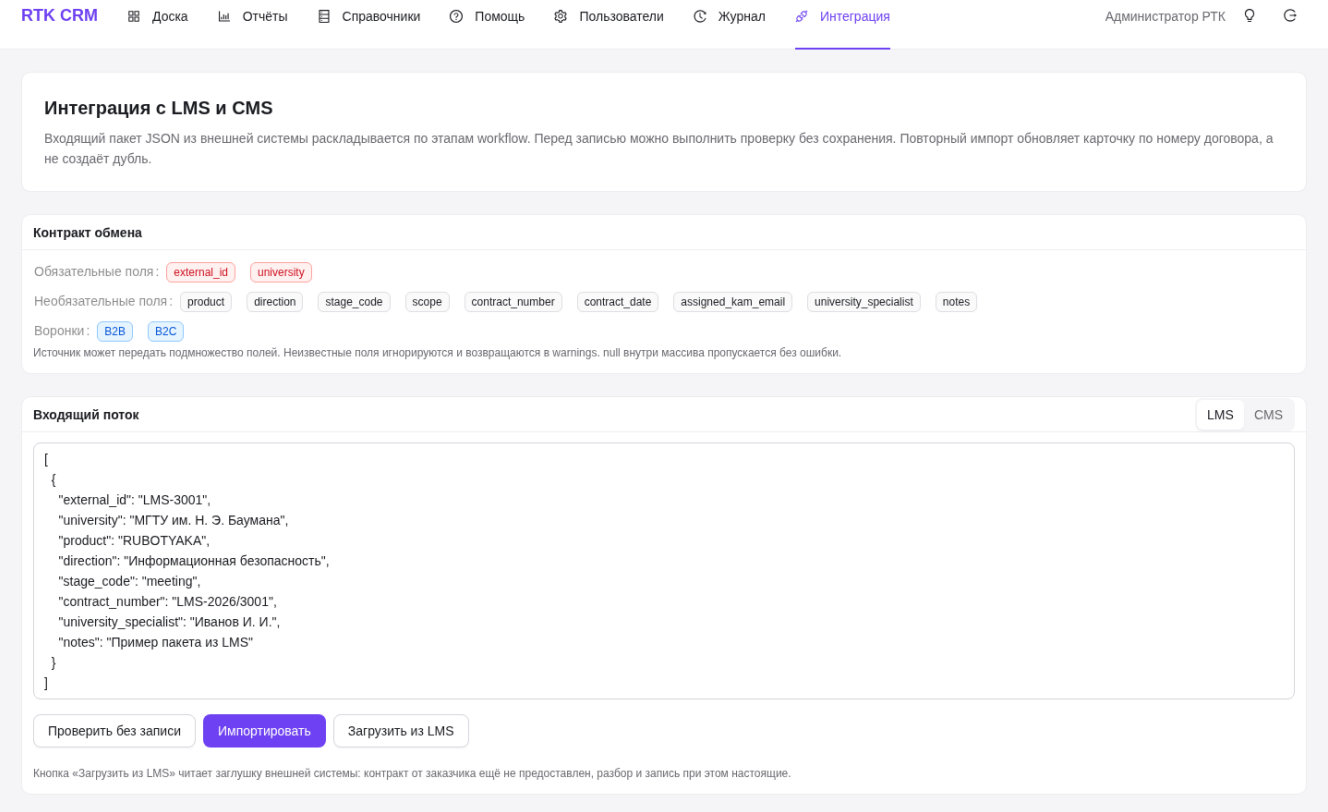


Рис. 7: Интеграция с LMS и CMS

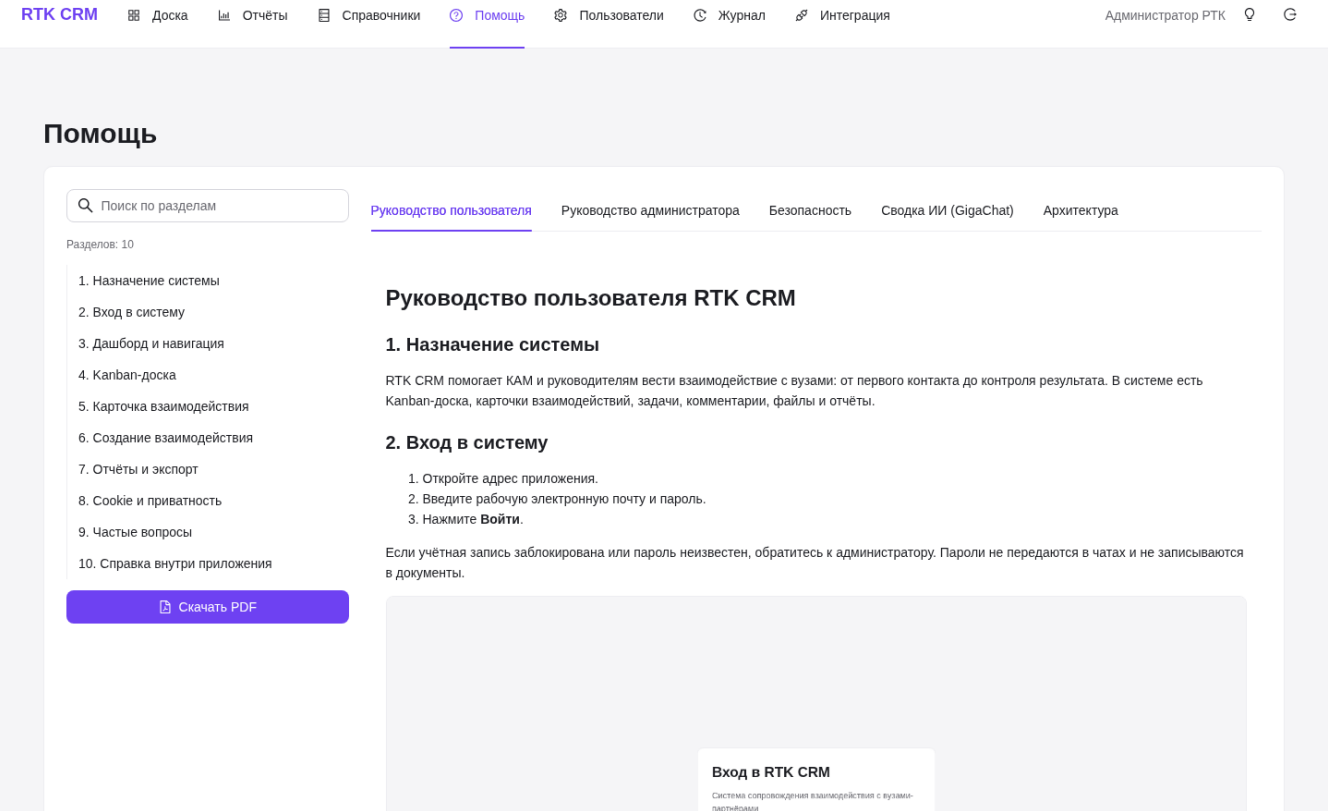


Рис. 8: Раздел «Помощь»

закрывается без удаления истории.

Создание: заполните ФИО, email, пароль и роль, затем «Добавить». Роль и активность можно менять прямо в списке; удаление доступно для любой учётной записи, кроме собственной.

Минимальные роли:

- **КАМ** — работа с взаимодействиями;
- **Руководитель** — просмотр отчётов, контроль процесса и назначение КАМ;
- **Администратор** — настройки и справочники.

Руководитель и администратор назначают ответственного КАМ при создании взаимодействия и могут передать существующую карточку другому сотруднику. КАМ работает только со своими взаимодействиями: чужие карточки недоступны по прямому id.

15.1.3 3. Этапы workflow

Откройте **Пользователи** и перейдите на вкладку **Этапы воркфлоу**. Конструктор показывает все этапы, включая выключенные. На каждой строке доступны сортировка, редактирование, переключатель активности и удаление. Переключатель **B2B / B2C** на той же вкладке задаёт воронку: у каждой свой набор этапов.

Порядок	Этап	Код	Взаимодействий	Действия
1	Поиск контактов ответственного в вузе	contact_search	1	☒
2	Коммуникация и уточнение актуальности программ	communication	0	☒
3	Организация встречи с представителями вуза	meeting	1	☒
4	Обмен пакетом документов для подписания	document_exchange	0	☒
5	Корректировка документов перед подписанием	document_revision	0	☒
6	Подписание документов	document_signing	1	☒

Рис. 9: Конструктор workflow

15.1.3.1 Включение и выключение Переключатель работает в обе стороны. При ошибке интерфейс откатывает оптимистичное изменение. Этап с активными взаимодействиями нельзя отключить: сначала перенесите взаимодействия либо удалите этап с выбором целевого этапа.

15.1.3.2 Переименование и сортировка Измените название в строке этапа и подтвердите сохранение. Для изменения порядка перетащите строку за область drag-and-drop.

15.1.3.3 Удаление с переносом Перед удалением система показывает количество активных и всех взаимодействий. Если данные есть, выберите целевой этап. Перенос выполняется до удаления исходного этапа.

15.1.4 4. Справочники

В разделе **Справочники** поддерживаются вузы, ИТ-направления и ИТ-продукты. Перед удалением проверьте связанные данные. Не помещайте персональные данные в поле комментария без правового основания.

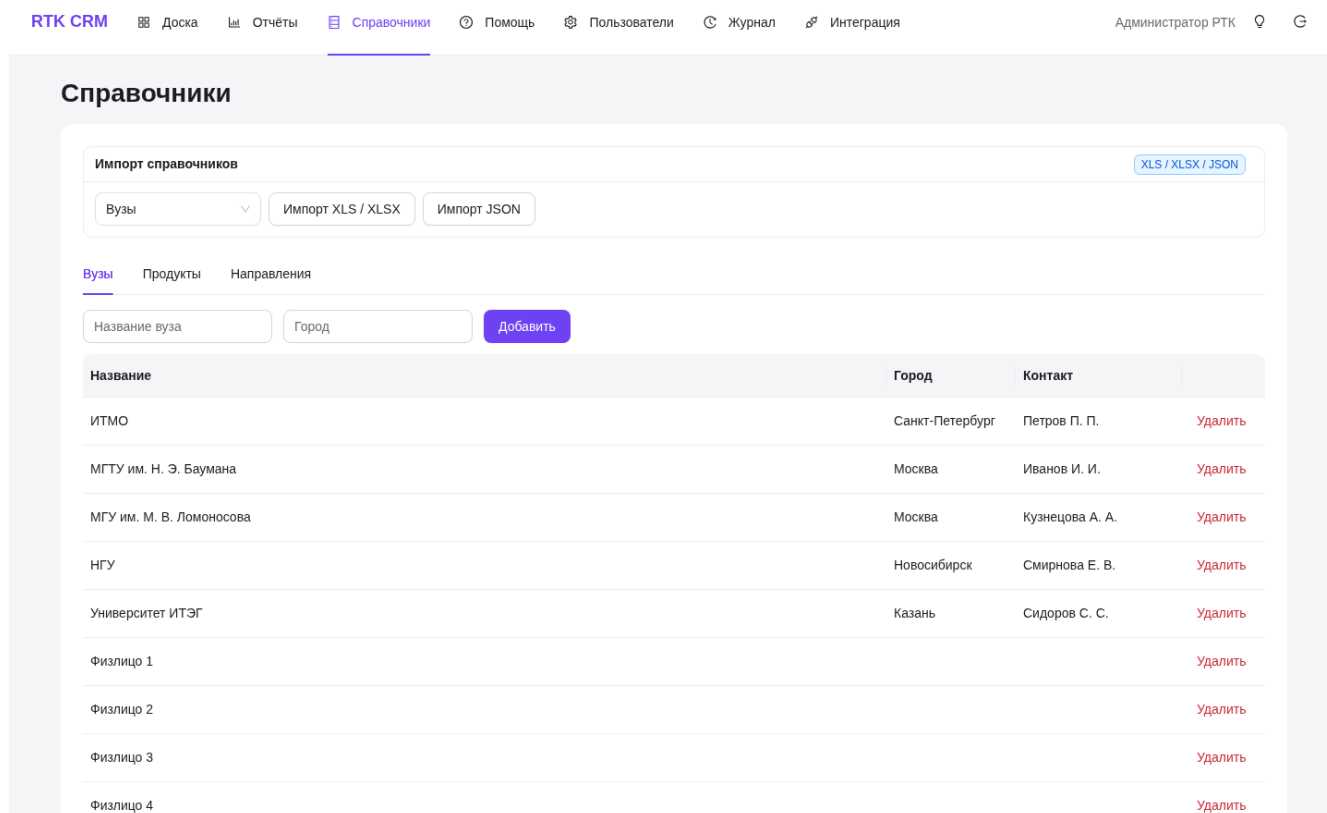


Рис. 10: Справочники

15.1.5 5. Импорт и экспорт

Экспорт отчётов выполняется на странице **Отчёты** в XLSX, XLS или PDF. Для массовой загрузки используйте импорт в разделе **Справочники**: диалог принимает XLS/XLSX и JSON, показывает предпросмотр и предлагает сопоставить колонки файла с полями справочника. Перед запуском сделайте резервную копию и проверьте сопоставление — строки с ошибками можно пропустить, импортировав только валидные.

15.1.6 6. Интеграция с LMS и CMS

Вкладка **Интеграция** описывает обмен с внешними системами заказчика. Администратор здесь проверяет контракт и управляет загрузкой пакетов.

Порядок действий:

Справочники

Импорт справочников

XLS / XLSX / JSON

Бузы

Импорт XLS / XLSX

Импорт JSON

Бузы

Продукты

Направления

Название вуза

Город

Добавить

Название	Город	Контакт	
ИТМО	Санкт-Петербург	Петров П. П.	Удалить
МГТУ им. Н. Э. Баумана	Москва	Иванов И. И.	Удалить
МГУ им. М. В. Ломоносова	Москва	Кузнецова А. А.	Удалить
НГУ	Новосибирск	Смирнова Е. В.	Удалить
Университет ИТЭГ	Казань	Сидоров С. С.	Удалить
Физлицо 1			Удалить
Физлицо 2			Удалить
Физлицо 3			Удалить
Физлицо 4			Удалить

Рис. 11: Импорт справочников из XLSX

1. Откройте **Интеграция** и посмотрите **схему обмена** — перечень полей контракта и обязательность каждого.
2. Вставьте пакет JSON в поле ввода либо нажмите **Загрузить пример**.
3. Нажмите **Предпросмотр**: пакет проверяется по контракту без записи в базу. Так проверяют формат перед реальным импортом (dry-run).
4. Нажмите **Импортировать**, если предпросмотр не показал ошибок.

Импорт идемпотентен по `contract_number`: повторная загрузка того же пакета обновляет существующее взаимодействие, а не создаёт второе. Это позволяет безопасно повторять обмен после сбоя сети.

Вкладка также формирует **исходящий пакет** — выгрузку для передачи во внешнюю систему в формате контракта.

В прототипе интеграция работает через `mosk`-адаптер: он возвращает детерминированные данные по тому же контракту. Подключение реальных LMS и CMS выполняется реализацией того же интерфейса, без изменений в остальной системе.

15.1.7 7. Аудит и безопасность

Изменения записываются middleware аудита (POST/PUT/PATCH/DELETE). Журнал открывается в разделе **Журнал**: время, сотрудник, действие, сущность и IP. Доступны фильтры по действию, сущности и сотруднику, сортировка — от свежих записей. При расследовании укажите пользователя, сущность, время и идентификатор операции. Не передавайте JWT, пароли и `.env` в тикетах или репозитории.

В рабочем контуре используйте HTTPS, ограниченный CORS, JWT/RBAC и минимальные права базы данных.

RTK CRM

Доска

Отчёты

Справочники

Помощь

Пользователи

Журнал

Интеграция

Администратор РТК

Интеграция с LMS и CMS

Входящий пакет JSON из внешней системы раскладывается по этапам workflow. Перед записью можно выполнить проверку без сохранения. Повторный импорт обновляет карточку по номеру договора, а не создаёт дубль.

Контракт обмена

Обязательные поля:

external_id

university

Необязательные поля:

product

direction

stage_code

scope

contract_number

contract_date

assigned_kam_email

university_specialist

notes

Воронки:

B2B

B2C

Источник может передать подмножество полей. Неизвестные поля игнорируются и возвращаются в warnings. pull внутри массива пропускается без ошибки.

Входящий поток

LMS

CMS

[

{

"external_id": "LMS-3001",

"university": "МГТУ им. Н. Э. Баумана",

"product": "РУБОТЫАКА",

"direction": "Информационная безопасность",

"stage_code": "meeting",

"contract_number": "LMS-2026/3001",

"university_specialist": "Иванов И. И.",

"notes": "Пример пакета из LMS"

}

]

Проверить без записи

Импортировать

Загрузить из LMS

Кнопка «Загрузить из LMS» читает заглушку внешней системы: контракт от заказчика ещё не предоставлен, разбор и запись при этом настоящие.

Рис. 12: Интеграция с LMS и CMS

15.1.8 8. Резервное копирование

Для Yandex Cloud ежедневный backup запускается `infra/yandex-cloud/backup.sh`. По умолчанию хранятся последние 7 дней. Периодически проверяйте восстановление на тестовой базе.

15.1.9 9. Устранение неисправностей

Пользователь не входит: проверьте активность учётной записи, CORS и состояние `backend /api/health`.

Экспорт не скачивается: убедитесь, что запрос выполняется из авторизованной сессии и в ответе присутствует `Content-Disposition`.

Этап не отключается: это защита при наличии активных взаимодействий.

Пустые графики: проверьте диапазон данных, активность этапов и ответ `GET /api/reports`.

16 Вопросы жюри и ответы

16.1 Вопросы жюри и ответы

Заготовка для защиты. Часть I — продуктовые вопросы, часть II — технические. Ответы короткие, со ссылками на файлы репозитория; места, где решение ещё не принято, помечены явно.

57

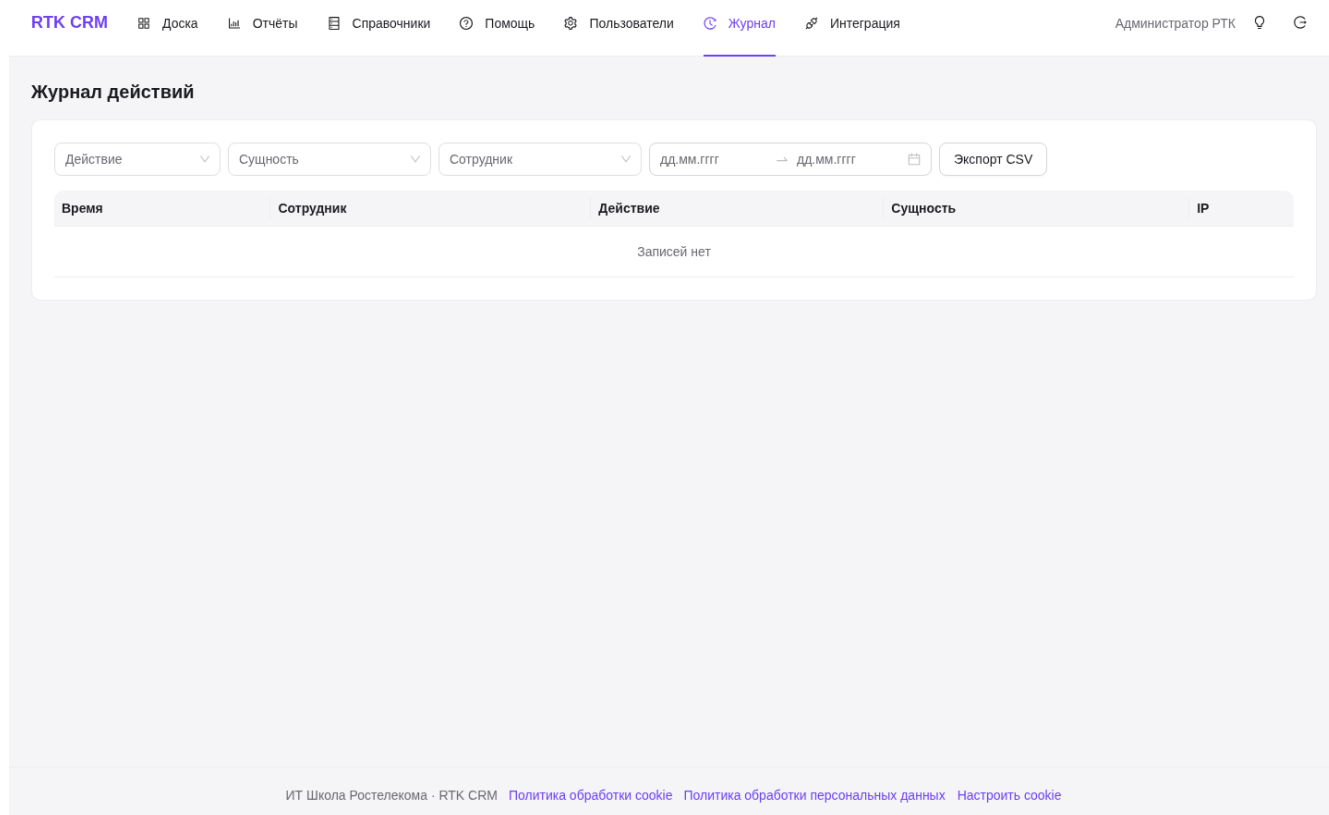


Рис. 13: Журнал аудита

16.1.1 Часть I. Продуктовые вопросы

16.1.1.1 1. Какую проблему решает система? Она объединяет в одном рабочем процессе взаимодействия с вузами, статусы, задачи, комментарии, документы и управленческие отчёты.

16.1.1.2 2. Кто основной пользователь? КАМ, который ведёт портфель вузов. Руководитель использует отчёты, а администратор управляет справочниками и workflow.

16.1.1.3 3. Почему Kanban? Kanban быстро показывает узкие места: количество карточек на каждом этапе, просроченные действия и следующий шаг по конкретному вузу.

16.1.1.4 4. Можно ли изменить 14 этапов? Да. Администратор может добавить, переименовать, отсортировать, включить, выключить или удалить этап с переносом взаимодействий.

16.1.1.5 5. Что произойдёт при отключении этапа с карточками? Backend отклонит опасную операцию. Сначала нужно перенести активные взаимодействия либо удалить этап с явно выбранным целевым этапом.

16.1.1.6 6. Как ограничивается доступ? JWT-аутентификация и RBAC проверяются на backend. Административные операции недоступны роли без соответствующего разрешения.

16.1.1.7 7. Где хранятся данные? Сейчас поддерживаются два контура. Основной — Yandex Cloud: VM, nginx, PostgreSQL и Object Storage, данные размещаются в РФ. Альтернатив-

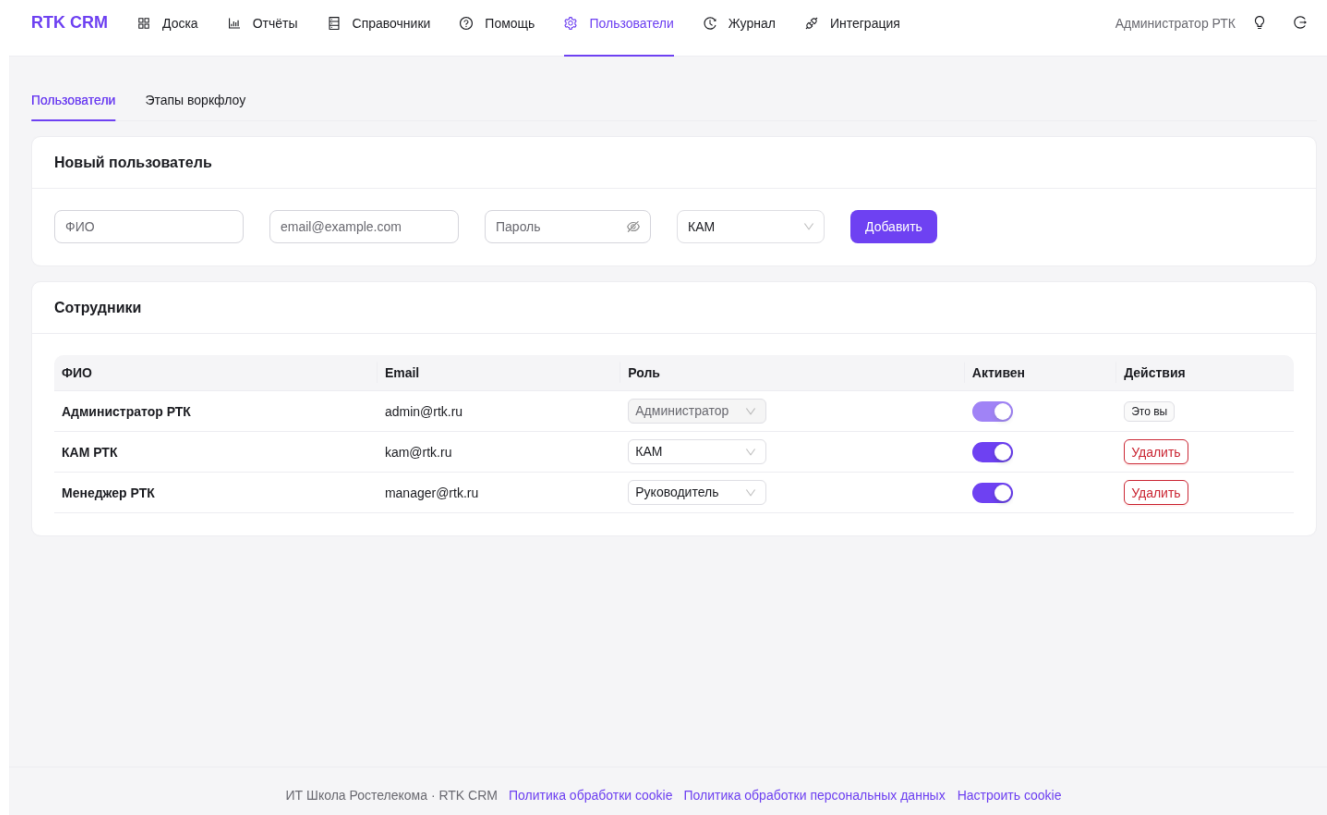


Рис. 14: Административные настройки

ный — Render (API) + Vercel (frontend) + Supabase (PostgreSQL), используется как публичный демонстрационный стенд. Код один и тот же, различия — в переменных окружения.

16.1.1.8 8. Почему основным контуром выбран Yandex Cloud? Требования 152-ФЗ к локализации баз данных граждан РФ и приказ ФСТЭК № 117 предполагают размещение в российском контуре. Yandex Cloud даёт аттестованную инфраструктуру, а весь стек уже контейнеризован, поэтому перенос выполняется сменой переменных окружения без правки кода.

16.1.1.9 9. Как обеспечивается 152-ФЗ? В продукте есть аудит, RBAC, HTTPS/CORS, политика приватности и cookie consent. Локализация данных требует отдельного подтверждения выбранного production-контура.

16.1.1.10 10. Какие данные считаются персональными? Контакты пользователей и представителей вузов, учетные данные и технические идентификаторы. Их нельзя помещать в тестовые данные и публичные скриншоты.

16.1.1.11 11. Как работает экспорт? Frontend выполняет авторизованный Axios-запрос как Blob, backend возвращает Content-Disposition, после чего браузер сохраняет XLSX, XLS или PDF.

16.1.1.12 12. Как проверяется производительность? В репозитории есть Locust-сценарий: логин, отчёты, Kanban и справочники. Целевые критерии — 50 пользователей, 60 секунд, p95 менее 1000 мс.

16.1.1.13 13. Есть ли уже фактические результаты нагрузки? Да, локальный прогон выполнен: 50 пользователей / 60 секунд, 1613 запросов, 0 ошибок, aggregate p95 93 мс; y /api/reports p95 7 мс. Отчёты лежат в tests/load/report.html и tests/load/report-10-reports.html. Это цифры локального стенда, а не подтверждённый SLA на ВМ заказчика — его нужно перепроверить на целевом контуре.

16.1.1.14 14. Что будет при недоступности backend? UI показывает загрузку или ошибку, не маскируя отсутствие данных. После восстановления соединения TanStack Query повторяет запрос.

16.1.1.15 15. Как система масштабируется? Frontend статичен, backend можно запускать несколькими экземплярами, а PostgreSQL масштабируется средствами выбранного managed-контура.

16.1.1.16 16. Почему FastAPI? Он даёт типизированные схемы, асинхронный стек и автоматическую OpenAPI- документацию при небольшом объёме инфраструктурного кода.

16.1.1.17 17. Как предотвращается потеря истории? Изменяющие операции проходят audit middleware, а workflow-операции сохраняют связь карточки с этапом и пользователем.

16.1.1.18 18. Как восстанавливаются данные? Для Yandex Cloud подготовлен ежедневный pg_dump с хранением семи последних копий. Восстановление должно регулярно проверяться на тестовой базе.

16.1.1.19 19. Что сделано для мобильных устройств? Метрики перестраиваются в сетку, формы становятся вертикальными, таблицы получают локальную горизонтальную прокрутку, а footer учитывает safe-area. Cookie-баннер на телефоне ограничен 55vh, кнопки встают столбиком, модалка настроек укладывается в экран.

16.1.1.20 20. Какой следующий шаг после защиты? Провести реальный нагрузочный тест, закрыть browser smoke test, подтвердить контур хранения данных и согласовать миграцию production в российское облако.

16.1.2 Часть II. Технические вопросы

16.1.2.1 1. Как работает двухсторонняя интеграция с LMS и CMS? Контракт обмена описан эндпоинтом GET /api/integration/schema, приём пакета — POST /api/integration/import, сухой прогон без записи — POST /api/integration/preview, выгрузка наружу — GET /api/integration/outbound (см. backend/api/integration.py). Повторный импорт сопоставляет карточку по номеру договора и обновляет её вместо создания дубля; неизвестные поля не ломают разбор, а возвращаются в warnings. Забор из внешней системы (GET /api/integration/pull/{source}) пока читает заглушку, потому что контракт LMS/CMS заказчиком не предоставлен — переход на живой источник сводится к замене источника данных.

16.1.2.2 2. Почему не версионизируете workflow? Версионирование не требовалось ТЗ, а изменение этапа действует на все активные карточки сразу — это осознанный выбор в пользу простоты поддержки администратором. Этапы разделены полем score на два набора: B2B (14 этапов ИТ-Школы) и B2C (упрощённая воронка), см. backend/models/entities.py. Историю изменений частично закрывает аудит: переход фиксируется записью в action_logs со старым и новым значением. Если версии понадобятся, следующий шаг — отдельная таблица версий workflow поверх текущей схемы.

16.1.2.3 3. Как реализован аудит-лог? Записи создаёт `middleware backend/middleware/audit.py` на каждый успешный изменяющий запрос (POST, PUT, PATCH, DELETE со статусом меньше 400); GET/OPTIONS/HEAD и эндпоинты логина и health пропускаются. В `action_logs` пишутся пользователь, действие (CREATE/UPDATE/DELETE), тип и id сущности, старое и новое значение и IP-адрес. Читается журнал администратором через GET `/api/audit` с фильтрами и выгрузкой в CSV (`backend/api/audit.py`) — данные можно не только накапливать, но и показать проверяющему.

16.1.2.4 4. Что с 152-ФЗ, где хранятся персональные данные? Персональными считаются контакты представителей вузов, учётные данные пользователей и технические идентификаторы; они не попадают в тестовые данные и публичные скриншоты. Основной контур — Yandex Cloud: VM, nginx, PostgreSQL и Object Storage в РФ, что закрывает локализацию данных граждан РФ (`docs/architecture/deployment-yandex-cloud.md`). Технические меры — RBAC, аудит, HTTPS/CORS и заголовки безопасности (`backend/middleware/security_headers.py`). Локализация конкретного production-контура требует отдельного подтверждения заказчиком, это вынесено в план следующего шага.

16.1.2.5 5. Как работает RBAC на уровне запросов? Роль берётся из JWT, а уровень доступа проверяется зависимостями FastAPI: `get_current_user`, `require_manager_or_admin`, `require_admin`. Точечные операции по id проходят через `backend/api/access.py`: KAM (роль user) видит только свои карточки, проверка идёт по `assigned_kam_id`, а чужой id даёт 403, а не открытие карточки. Логика вынесена в один модуль, чтобы одинаково применяться и во взаимодействиях, и в файлах.

16.1.2.6 6. Как обрабатываете 50 параллельных пользователей? Сценарий — `tests/load/locustfile.py`: логин, отчёты, Kanban и справочники. Прогон выполнен локально, отчёты лежат в `tests/load/`: **50 пользователей / 60 секунд, 1613 запросов, 0 ошибок, aggregate p95 93 мс**. Отдельный профиль `ReportsUser` дал 322 запроса, 0 ошибок, `y /api/reports` p95 7 мс. Под эту нагрузку заложены асинхронный стек (FastAPI + SQLAlchemy async), кэш агрегатов отчётов на 30 секунд (`backend/services/report_cache.py`) и индексы на фильтруемых полях карточек. Production SLA отдельно не подтверждался — это цифры локального прогона, а не гарантия для VM заказчика.

16.1.2.7 7. Что с утечками, снимали ли профайлер? Профайлер памяти не запускался, и объявлять его результаты мы не будем — это честная позиция. Что уже ограничивает утечки: тело запроса для аудита обрезается, состояние сессии БД живёт внутри запроса (`SessionLocal` как контекстный менеджер), глобального изменяемого состояния на запрос нет, а кэш отчётов имеет TTL. Следующий шаг — прогон `tracemalloc.py-spy` под нагрузкой и фиксация baseline в отчёте.

16.1.2.8 8. Как устроен импорт XLS/XLSX, где диалоговый маппинг? Импорт каталогов — `backend/services/excel_import.py`: `openpyxl` для XLSX, `xlrd` для XLS, структура проверяется `jsonschema`. Диалоговый маппинг колонок есть в UI (`frontend/src/pages/DirectoryPage.tsx`): после выбора файла открывается модалька «Сопоставление полей», пользователь видит до пяти предпросмотренных строк и сопоставляет колонки, а выбранная схема сохраняется в `localStorage` по типу справочника и подставляется при следующем импорте. Проверка идёт на backend через `preview`, обязательные поля нельзя оставить несопоставленными. Ошибки возвращаются по строкам (`ValidationIssue`: строка, поле, проблема, уровень — `error` блокирует строку, `warning` нет), а отчёт о них выгружается в XLSX (`backend/services/import_report.py`).

16.1.2.9 9. Как тестировали write-операции? Изменяющие сценарии покрыты интеграционными тестами, которые ходят через API в реальную БД: `test_audit_log.py`, `test_interaction_access.py`, `test_user_delete.py`, `test_workflow_scope.py`, `test_integration.py`,

test_excel_import.py, test_json_export.py. Всего в наборе 200 проходящих тестов, отдельно 22 теста Keycloak. Моки не применяются: проверяются реальные пути записи, ограничения целостности и запись в аудит.

16.1.2.10 10. Что с масштабированием на 300+? Backend без собственного состояния, поэтому масштабируется репликами за балансировщиком; сессии, файлы и кэш вынесены в PostgreSQL, S3 и Redis/KeyDB (docs/architecture/ARCHITECTURE.md, раздел «Масштабируемость»). Что уже заложено в код: пул соединений, индексы под фильтрацию доски и отчётов, кэш агрегатов на 30 секунд. При росте нагрузки следующий шаг — реплики чтения PostgreSQL и вынос отчётов на отдельный пул.

16.1.2.11 11. Почему KeyDB, а не Redis? Сейчас в контуре Redis 7, а KeyDB — многопоточный форк Redis с тем же протоколом, рекомендованный заказчиком как более лёгкий (docs/STACK.md). Переход не требует правок кода: адрес задаётся переменной окружения redis_url, а backend/services/report_cache.py работает через обычный клиент Redis. То есть это замена эксплуатационного слоя, а не переписывание логики.

16.1.2.12 12. Как обрабатываете удаление статуса с активными задачами? Удаление этапа — DELETE /api/stages/{stage_id} с необязательным target_stage_id (backend/api/stages.py). На уровне БД стоит ON DELETE RESTRICT, поэтому активный этап нельзя удалить, пока на нём есть карточки: без target_stage_id вернётся 409 с количеством взаимодействий, а с ним карточки переносятся на целевой этап и только потом этап удаляется. Это защищает от потери задач при чистке воронки.

16.1.2.13 13. Как реализован экспорт в JSON? GET /api/reports/json (backend/api/reports.py) отдаёт воспроизводимую выгрузку: generated_at, total, описание колонок из общего конфига config/report_columns.json, блок storage (s3 или local и имя бакета), применённые фильтры и сами взаимодействия с ключами файлов. По требованию Q&A Крылова выгрузка содержит не только данные карточек, но и ключи связи, чтобы принимающая сторона сопоставила объекты и скачала вложения. Колонки и фильтры те же, что в интерфейсе и в PDF/XLS-выгрузках.

16.1.2.14 14. Что с LLM — как обоснуете использование? LLM применяется только для одной необязательной функции — сводки по карточке (ФТ-6), см. backend/services/gigachat.py. Без ключа GIGACHAT_CREDENTIALS функция выключена и отдаёт 503, а интерфейс скрывает кнопку; ошибки внешнего сервиса не превращаются в 500. В промпт уходят только тексты комментариев, названия задач, вуза и продукта — персональные данные исключены, а демо-сводка без обращения к модели явно помечается отдельным значением поля «Модель».

16.1.2.15 15. Как деплоите — Docker, Kubernetes? Основной способ — Docker и Docker Compose: базовый docker-compose.yml и прод-стек infra/yandex-cloud/docker-compose.prod.yml с nginx (docs/DEPLOYMENT.md). Kubernetes в текущем контуре не используется; он указан как путь горизонтального масштабирования, когда одной ВМ перестанет хватать. Такой выбор оправдан размером нагрузки и сроком: одна ВМ с контейнерами проще в поддержке, а переход на оркестратор не требует правок кода.

16.1.2.16 16. Что с целостностью данных при параллельном обновлении? Транзакции идут через сессию SQLAlchemy, а критичные инварианты проверяются на сервере: лимит активных взаимодействий на вуз _ensure_parallel_limit (backend/api/interactions.py), совпадение score карточки и этапа (409), уникальность этапов по (scope, code) и (scope, order). Явного оптимистичного блокирования (колонки версии или SELECT FOR UPDATE) сейчас нет — при одновременной правке одной карточки действует «последняя запись побеждает», и это задокументированный следующий шаг. Предлагаемое решение — колонка version или блокировка строки при конфликтных операциях.

16.1.2.17 17. Как считаете ранжирование программ? Отдельного алгоритма ранжирования программ в системе нет, и мы не будем его изображать. Отчёты дают агрегаты и фильтры по этапам, вузам, продуктам и направлениям, а сортировка выполняется по колонкам выгрузки (`config/report_columns.json`). Если заказчику нужен приоритетный список, это формулируется как отдельная задача с явными весами критериев, а не подразумевается текущей логикой.

16.1.2.18 18. Почему Python, а не Kotlin? ТЗ само предлагает Python и PostgreSQL, и FastAPI закрывает задачу без выигрыша от смены языка: асинхронный ввод-вывод под нагрузку 300+, типизированные схемы и общий язык типов с Pydantic, автоматическая OpenAPI-документация (`docs/architecture/ARCHITECTURE.md`). Kotlin потребовал бы отдельной команды и переписывания без преимуществ для задач CRM. Команда владеет Python, поэтому скорость разработки и поддержки выше.

16.1.2.19 19. Что сделано для мобильных устройств? Интерфейс адаптивный: метрики перестраиваются в сетку, формы становятся вертикальными, таблицы получают локальную горизонтальную прокрутку, а footer учитывает `safe-area`. Cookie-баннер на телефоне ограничен 55vh со скроллом внутри, кнопки встают столбиком на всю ширину, а модалка настроек укладывается в экран и не перекрывает свой футер. Отдельного нативного приложения нет — сценарий KAM закрывается браузером на телефоне. Если потребуется, следующий шаг — PWA-оболочка поверх текущего SPA.

16.1.2.20 20. Что с документацией — встроена ли в платформу? Да, документация встроена: вкладка «Помощь» (`/help`) отдаёт Markdown из статического зеркала `frontend/public/docs/`, которое собирается скриптом `scripts/sync_public_docs.py`. Рядом лежат единый PDF (`docs/rtk-crm-documentation.pdf`), отдельные руководства пользователя и администратора и архитектурные схемы. Зеркало проверяется тестом на дрейф (`tests/test_docs_mirror.py`), поэтому «Помощь» не расходится с исходниками.

17 Сценарии работы

17.1 КАМ

1. Вход, доска с карточками своих вузов.
2. Открытие карточки, постановка задачи со сроком, комментариев.
3. Перемещение карточки по этапам.
4. Запрос ИИ-сводки по взаимодействию.
5. Отчёт по своим вузам и экспорт в XLSX/PDF.

17.2 Администратор

1. Управление пользователями и ролями.
2. Настройка этапов воркфлоу: порядок, цвет, включение.
3. Импорт справочников из XLSX с сопоставлением колонок.
4. Просмотр журнала аудита.
5. Настройка интеграции с LMS/CMS: предпросмотр и импорт пакета.

18 Глоссарий

Термин	Определение
Взаимодействие	Связка «вуз + ИТ-продукт», основная сущность системы
Этап	Стадия воронки, по которой перемещается взаимодействие
КАМ	Клиентский менеджер, ответственный за взаимодействие
Вуз	Университет-партнёр ИТ Школы
Контракт обмена	Схема JSON-пакета для интеграции с LMS/CMS
Идемпотентный импорт	Повторный импорт не создаёт дублей
Dry-run	Проверка пакета без записи в базу
RBAC	Ролевое разграничение доступа
PKCE	Расширение Authorization Code Flow для публичных клиентов

19 Приложения

19.1 Скриншоты интерфейса

Актуальные снимки экранов (светлая и тёмная темы, десктоп и мобильный) лежат в docs/images/ и зеркалируются в frontend/public/docs/images/: вход, доска, карточка, фильтры, справочники, импорт, отчёты, конструктор воркфлоу, пользователи, журнал, интеграция, помощь.

19.2 Примеры отчётов

Образцы выгрузок: docs/examples/report-b2b-2026-09-24.pdf, docs/examples/report-longnames-2026-09-24.pdf.

19.3 Диаграммы

Исходники ArchiMate (компонентная, функциональная, ER) — в docs/architecture/; C4-схемы — там же в Markdown.

Дата сборки: 2026-09-27. Версия документа: 2026-09-27.